

Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory

(CSE20110)

Laboratory Continuous Assessment 3

Campus Connect: Campus Chat and File Sharing System

Submitted by

Sr. No.	Student Name	PRN
1	Harsh Patil	Not shown

Group No. 1 Panel Batch

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Academic Year 2026–2027

Submission Date

Department of Computer Engineering and Technology

CERTIFICATE

This is to certify that the mini-project entitled “Campus Connect: Campus Chat and File Sharing System” has been carried out by the following students of Third Year B.Tech. Computer Science and Engineering, Panel , Batch , Group No. 1, under my guidance during the academic year 2026–2027.

Student Name	**PRN**
Harsh Patil	Not shown

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature _____

Project Guide and Course Faculty

Date _____ Place: Pune

Acknowledgement

I thank the faculty of the Computer Networks Laboratory and the Department of Computer Engineering and Technology, MIT World Peace University, Pune, for the course, the mini-project template and the assessment framework within which this work was done. The project made use of the public documentation of Node.js, Express and Socket.IO and of the IETF standards listed in the References. An AI assistant (Claude, from Anthropic) helped with the code, the tests and the drafting of this report, as described in the AI Usage Declaration.

Group No. 1

Abstract

Students often share notes and project files through general-purpose messaging apps, which give the college no view of how the data moves. This mini project builds Campus Connect, a web-based chat and file-sharing system, to study client–server communication in practice. A Node.js server listens on TCP port 3000 and handles two kinds of traffic: HTTP requests for sign-in, history and file transfer, and Socket.IO events over a WebSocket connection for live messages, typing and online status. The system falls back to HTTP long-polling when WebSocket is not used. Session, membership and upload rules protect every request, and a built-in monitor reports connections, transports, bytes and server-measured round-trip time. An automated suite of 70 checks passed, covering normal use, invalid input, refused access, fallback, reconnection and restart. On one computer over the loopback interface, the median message delivery time was 1.65 ms over WebSocket and 4.90 ms over long-polling; these figures exclude real network delay. Public hosting was attempted but not completed, so the system was tested on localhost only. The project shows how HTTP, WebSocket and TCP work together in a real application.

Keywords: WebSocket, Socket.IO, client–server architecture, TCP

Table of Contents

1 Introduction and Project Objectives....7
1.1 Introduction and Background....7
1.2 Problem Statement....7
1.3 Scope....8
1.4 Objectives....8
2 Proposed Design and Requirements....9
2.1 Architecture and Network Topology....9
2.2 Tools and System Requirements....10
2.3 Addressing and Protocol Selection....10
3 Implementation and Technical Working....12
3.1 Methodology....12
3.2 Configuration or Code Implementation....12
3.3 Protocols and Technical Working....13
3.4 Execution Procedure....15
4 Testing and Results....16
4.1 Test Plan....16
4.2 Results and Execution Evidence....17
4.3 Objective Achievement....20
5 Discussion and Conclusion....21
5.1 Discussion and Comparative Analysis....21
5.2 Advantages and Applications....21
5.3 Limitations....22
5.4 Challenges and Troubleshooting....22
5.5 Future Scope....23
5.6 Conclusion....23
References....24
AI Usage Declaration....25
Appendices....26
Appendix A Code and Configuration....26
Appendix B Additional Evidence....40
Appendix C Individual Contributions....44

List of Figures

Figure 2.1 Proposed architecture and network topology....9
Figure 2.2 Protocol layers used by Campus Connect....11
Figure 3.1 Message sequence for a chat message and a shared file....12
Figure 3.2 Implementation evidence: a chat with a shared file....15
Figure 4.1 Test execution result (selected groups of npm test)....17
Figure 4.2 Protocol evidence from the running server....18
Figure 4.3 Network monitor page with live measurements....19
Figure 4.4 Message delivery time: WebSocket compared with HTTP long-polling....20
Figure B.1 Activity page showing connection upgrades and transfers....41
Figure B.2 Browser while the connection to the server was cut....41
Figure B.3 Browser after the connection returned....42

List of Tables

Table 1.1 Project objectives and verification methods....8
Table 2.1 Software, hardware and tools used....10
Table 2.2 Addressing used in the tests....10
Table 2.3 Protocols, ports and message formats....11
Table 3.1 Socket.IO events used by Campus Connect....14
Table 4.1 Test plan and results....16
Table 4.2 Message and request timings on the loopback interface (milliseconds)....19
Table 4.3 File transfer timings on the loopback interface (median of 3 runs, milliseconds)....19
Table 4.4 Objective achievement....20
Table 5.1 WebSocket compared with HTTP long-polling (measured, milliseconds)....21
Table 5.2 Challenges and troubleshooting....22
Table B.1 TCP sockets involving port 3000 during the handshake test....40
Table B.2 Complete speed-test statistics (milliseconds)....40
Table C.1 Individual contributions....44

1 Introduction and Project Objectives

1.1 Introduction and Background

Chat and file sharing are everyday campus services, and they rest on several ideas studied in this laboratory. A web page is normally delivered by HTTP, a request–response protocol: a client sends a request and the server answers with one or more responses [1]. HTTP runs on TCP, which gives applications a reliable, in-order byte stream and uses port numbers to tell services apart [2]. This model suits page loads but not chat. The server cannot start the conversation, so a browser has to keep asking whether anything new has arrived.

Two techniques remove that limit. In long-polling, the client keeps a request open until the server has something to send. The WebSocket protocol starts as an ordinary HTTP GET with an Upgrade header and a random Sec-WebSocket-Key. The server answers 101 Switching Protocols with a Sec-WebSocket-Accept value worked out from that key, and the same TCP connection then carries messages in both directions [3]. Socket.IO is a library built on this idea. Its lower layer, Engine.IO, starts with HTTP long-polling, upgrades to WebSocket when it can, and sends a ping at a fixed interval; if no pong comes back within the timeout, the connection is treated as closed. Its upper layer adds acknowledgements, rooms and automatic reconnection [4].

Three sources shaped the project. RFC 9293 [2] states the service that sits under both HTTP and WebSocket: reliable, in-order delivery between ports. RFC 6455 [3] defines the handshake that this report reproduces by hand in Section 4.2. The Socket.IO documentation [4] describes the polling-first sequence and the heartbeat that the project configures and tests.

1.2 Problem Statement

Students share notes, assignments and project files through general-purpose messaging apps. The college cannot see or control how that traffic behaves, and in a networking course the protocols stay hidden inside a finished product. The problem addressed here is to build a small campus chat and file-sharing application in which each networking step is visible and measurable: which transport carries a message, how long delivery takes, how many bytes move, and what happens when a connection drops or a request is not allowed. A service that accepts connections from many students must also refuse unauthorised access and bad input instead of trusting what it receives.

1.3 Scope

The users are registered students of one campus who use a web browser. The application is one Node.js server process with a SQLite database; it offers one-to-one chats, group chats, file sharing, search, an activity log and a Network monitor page. The work assumes a current browser and a host that runs Node.js 22.13 or newer.

The boundaries are as follows. All tests and measurements were made on one computer over the loopback interface (127.0.0.1), with two browser sessions and scripted clients. The largest group tested had one sender and ten receivers. The system was not deployed to a public host, so it was not tested on a real LAN, over Wi-Fi or over the Internet. The server itself speaks plain HTTP; HTTPS would be added by a hosting proxy. Voice, video and end-to-end encryption are outside the scope.

1.4 Objectives

The objectives in Table 1.1 can each be checked by a test or by recorded evidence. Their results are collected in Section 4.3.

Table 1.1 Project objectives and verification methods

Objective ID	Objective	Verification Method
O1	Serve the web application from a TCP server on port 3000 and keep accounts, chats and files after a restart.	Health check; restart checks (T1, T12); Table 2.2
O2	Deliver chat messages in real time over WebSocket, in one-to-one and group chats, with acknowledgements.	Live messaging checks (T6–T8); timings in Table 4.2
O3	Share files over HTTP with a live notice to the recipient, and return byte-identical files.	File checks (T9, T10); byte comparison in the speed test (T16)
O4	Fall back to HTTP long-polling when WebSocket is not used, and reconnect after a dropped connection.	Fallback and reconnection checks (T11); polling timings in Table 4.2
O5	Refuse requests that are not signed in, come from another website, or break the rules for input, files and sign-in attempts.	Protection checks (T2–T5, T7, T10, T13); Figure 4.2
O6	Measure connections, transports, bytes and round-trip time inside the application, and compare WebSocket with long-polling.	Network monitor checks (T15); speed test (T16); Figures 4.3 and 4.4
O7	Make the application reachable on a public address.	Opening the public address from another device

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

Campus Connect follows the client–server model: one server process holds all shared state, and each student reaches it through a browser. Figure 2.1 shows the setup used for every test in this report. One computer runs the server and the clients, which are two browser sessions that play the two students plus scripted clients that behave like browsers. All traffic stays on the loopback interface, so no physical link, switch or router is involved. The lower strip shows how the same server would be reached after hosting, with a proxy adding HTTPS; that setup was prepared but not deployed (Section 4.3).

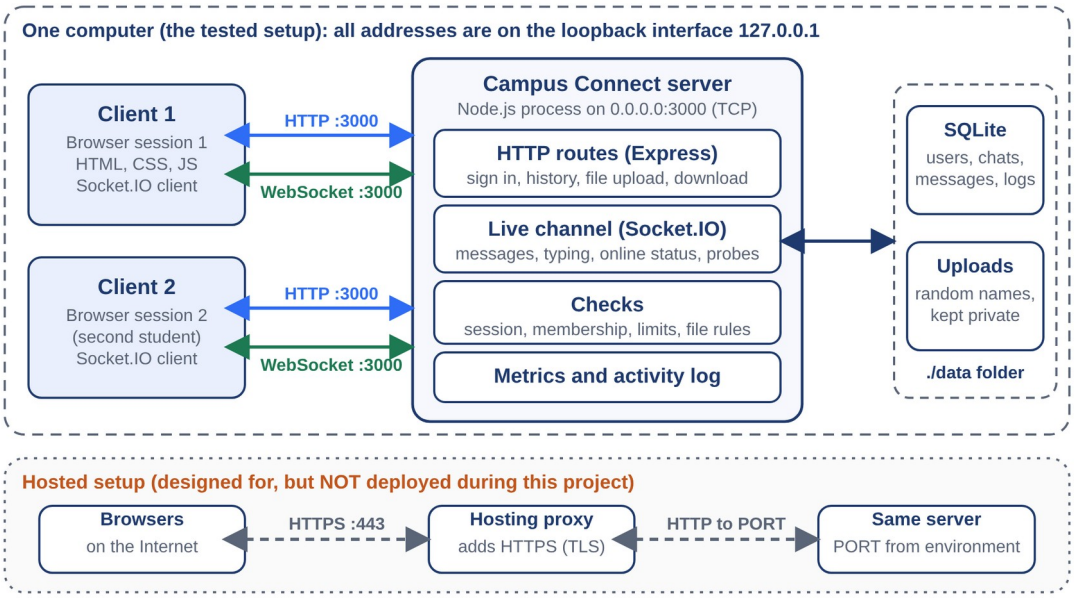


Figure 2.1 Proposed architecture and network topology

Each client opens two kinds of TCP connection to the same server port. Short HTTP requests load the pages and carry sign-in, chat history, and file upload and download. One long-lived Socket.IO connection starts as HTTP and is then upgraded to WebSocket [3], [4]; it carries live messages, typing signals, online status and the server's latency probes.

The server has four parts. The HTTP routes (Express) answer API requests and serve the pages. The live channel (Socket.IO) manages connections, rooms and events. A checks layer applies the same rules to every request: a signed-in session, chat membership, message and file limits, and an Origin check against other websites. A metrics module counts connections, messages, bytes and round-trip times and keeps an activity log. Data lives under the `data` folder: a SQLite database and an `uploads` folder where files are stored under random names.

Data flow for a message: the sender's browser emits `message:send` over WebSocket; the server checks the session and chat membership, saves the message, acknowledges it, and emits `message:new` to every member of the chat. For a file: the browser sends an HTTP POST, the server checks size, type and first bytes and stores the file, and the recipients get a `message:new` event with the file name and size; their browsers then fetch the file with an HTTP GET (Figure 3.1).

2.2 Tools and System Requirements

Table 2.1 lists what was used, with the versions installed when the tests ran on 21 October 2026.

Table 2.1 Software, hardware and tools used

Tool or Resource	Version or Specification	Purpose
Test computer	Linux 6.18 (x86-64), 2 CPU cores, 8 GB RAM	Ran the server, the tests, the speed test and the browser sessions
Node.js	22.22.0 (project needs 22.13 or newer)	JavaScript runtime for the server
SQLite (`node:sqlite`)	Built into Node.js 22; experimental when it left its flag in v22.13.0 [5]	Database file for users, chats, messages and file records
Express	5.3.0	HTTP routes and static pages
Socket.IO	4.8.4 (the client, socket.io-client, is used only in tests)	Live channel: WebSocket with long-polling fallback, acknowledgements, rooms
bcryptjs, helmet	3.0.3, 8.3.0	Password hashing (cost factor 10); security headers including Content-Security-Policy
express-rate-limit, multer, cookie, Chart.js	8.7.1, 2.4.0, 2.0.1, 4.5.1	Sign-in limit, multipart upload reading, session cookie reading, charts on the Network page
Chromium (headless), curl, a Node.js TCP script	Chromium driven by Playwright; curl from the system	Screenshots, the offline check and the protocol evidence in Figure 4.2

2.3 Addressing and Protocol Selection

This is a socket-style project, so Table 2.2 gives the addresses and ports used in the tests and Table 2.3 the protocols and message formats. The gateway is "Not applicable" because client and server share one machine, and the hosted address is "Not assigned" because hosting was not completed. Figure 2.2 places these protocols in the TCP/IP layers, and the bullets after it give the design decisions.

Table 2.2 Addressing used in the tests

Device / Interface	IP / Prefix	Gateway	Role
Server listening socket	0.0.0.0, TCP port 3000 (the `PORT` variable can change it)	Not applicable	Accepts HTTP and WebSocket connections
Loopback interface	127.0.0.1 (127.0.0.0/8)	Not applicable	Path for all tested traffic

Device / Interface	IP / Prefix	Gateway	Role
Client sockets	127.0.0.1, source port chosen by the operating system (for example 41390)	Not applicable	TCP clients (browsers and test scripts)
Hosted public address	Not assigned	Not applicable	Would be given by the host, with HTTPS on port 443 forwarded to `PORT`

Table 2.3 Protocols, ports and message formats

Protocol	Transport and port	Message format and use
HTTP/1.1	TCP 3000	JSON bodies for sign-in, chats and history; `multipart/form-data` for file upload; file download as a binary attachment
WebSocket (RFC 6455) carrying Socket.IO	TCP 3000, upgraded from HTTP	Socket.IO packets: an event name plus JSON data, for example `message:send`; at most 100,000 bytes per message
HTTP long-polling (fallback)	TCP 3000, path `/socket.io/`	The same Socket.IO packets, sent in the bodies of GET and POST requests

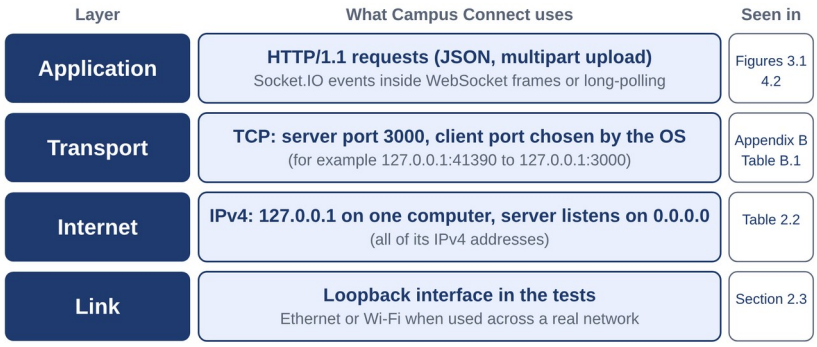


Figure 2.2 Protocol layers used by Campus Connect

- **TCP and HTTP.** Chat text and files must arrive complete and in order, which TCP provides [2]. Sign-in, history and files are request–response jobs that suit HTTP, and browsers already handle cookies, uploads and downloads.
- **WebSocket for live messages.** One long-lived TCP connection carries traffic both ways, so the server can push a message at once without polling requests [3]. Socket.IO adds acknowledgements, rooms and reconnection [4].
- **Long-polling as a fallback.** Both transports stay enabled, as in the Socket.IO default [6], so the chat still works where WebSocket is blocked.
- **Tighter limits than the defaults.** The heartbeat is 10 s with an 8 s timeout (defaults 25 s and 20 s), so a dead connection is noticed sooner. The Socket.IO message limit is 100 KB (default 1 MB) [6], enough for a 2,000-character message; files go over HTTP, up to 20 MB.

3 Implementation and Technical Working

3.1 Methodology

The system is organised in six parts, and each part is covered by checks in the automated suite described in Chapter 4.

1. **Server and pages.** Express on a Node.js HTTP server, with a health route and security headers.
2. **Accounts and sessions.** Registration, hashed passwords and a random session token in a cookie.
3. **Storage.** A SQLite database and an uploads folder under `data`.
4. **Live channel.** Socket.IO with a session check, one room per chat, acknowledged messages, typing and online status.
5. **File sharing.** HTTP upload with type, size and content checks, a live notice and a protected download.
6. **Measurement.** Counters, round-trip probes, an activity log, the Network page and a speed-test script.

Figure 3.1 shows the order of messages for the two main workflows. Steps 1 to 4 send a chat message over WebSocket. Steps 5 to 9 send a file: the file bytes travel over HTTP, and only the short notice travels over WebSocket.

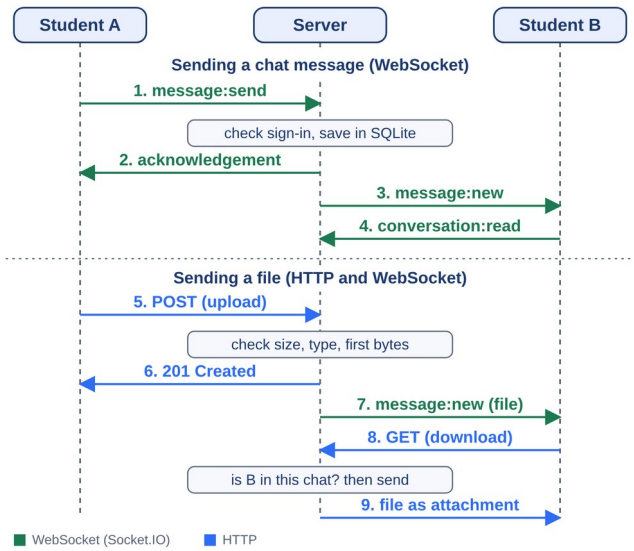


Figure 3.1 Message sequence for a chat message and a shared file

3.2 Configuration or Code Implementation

The code is split into a few modules. `server.js` holds the HTTP routes, the live channel and the start-up code. `src/auth.js` handles passwords and sessions, `src/db.js` the database,

`src/files.js` the file rules and `src/metrics.js` the counters and the activity log. The pages are in `public/`, the 70 checks in `tests/run.js` and the speed test in `scripts/bench.js`. Two short excerpts are given below; the full server code is in Appendix A.

server.js: live channel set-up and sign-in check

```
const io = new Server(server, {
  pingInterval: 10000,
  pingTimeout: 8000,
  maxHttpBufferSize: 1e5,
  allowRequest: (req, cb) => cb(null, !req.headers.origin || sameHost(req.headers.origin,
req.headers.host)),
});

io.use((socket, next) => {
  const user = auth.userFromToken(auth.tokenFromCookieHeader(socket.request.headers.cookie));
  if (!user) return next(new Error('unauthorized'));
  socket.data.user = user;
  next();
});
```

The first block sets the heartbeat and size limits discussed in Section 2.3. `allowRequest` accepts a connection only when it has no Origin header or the Origin matches the server's own host, which stops other websites from opening the live channel. The `io.use` function runs for every new connection and refuses visitors who have no valid session cookie.

server.js: handling a message (lines omitted are marked)

```
socket.on('message:send', (payload, ack) => {
  const reply = typeof ack === 'function' ? ack : () => {};
  try {
    const cid = Number(payload && payload.conversationId);
    const body = typeof (payload && payload.body) === 'string' ? payload.body.trim() : '';
    if (!Number.isInteger(cid) || !isMember(cid, user.id)) return reply({ ok: false, error: 'Conversation
not found.' });
    if (!body) return reply({ ok: false, error: 'Type a message first.' });
    if (body.length > 2000) return reply({ ok: false, error: 'Messages can be up to 2000 characters.' });
    ...  (flood limit: at most 15 messages per 10 seconds per connection)
    const { msg, deliveries } = postMessage({ cid, userId: user.id, body });
    ...  (counters and activity log)
    reply({ ok: true, message: msg });
  } catch (err) { ... }
});
```

Every check happens on the server, whatever the browser does. A student outside the chat gets the same "Conversation not found" answer as for a chat that does not exist, and the acknowledgement carries either the saved message or an error text.

3.3 Protocols and Technical Working

Connection set-up. A student signs in with `POST /api/auth/login`. The server checks the password against its bcrypt hash and answers with a `Set-Cookie` header that holds a random session token (HttpOnly, SameSite=Lax, and Secure in production). The page then opens the Socket.IO connection. The first request is an HTTP GET to `/socket.io/?EIO=4&transport=polling`; its reply holds a session id, the allowed upgrade (`websocket`) and the heartbeat values (Figure 4.2). The client then sends an HTTP upgrade request, the server

answers `101 Switching Protocols`, and the same TCP connection carries WebSocket frames [3]. Before the connection is accepted, the `io.use` check reads the cookie. The accepted connection joins the rooms `user:<id>`, `feed` and one `conv:<id>` for each of the student's chats.

Message exchange. Table 3.1 lists the events. Messages are sent with an acknowledgement; the server saves the message and then emits `message:new` to the chat's room, so every member who is online receives it at once. A member who is offline reads it later from the stored history.

Table 3.1 Socket.IO events used by Campus Connect

Event	Direction	Purpose
`message:send`	Client to server, acknowledged	Send a chat message; reply is `{ok, message}` or `{ok:false, error}`
`message:new`	Server to a chat room	A new message or file notice reaches every member
`conversation:read`	Client to server	Mark messages as read, which clears the unread count
`conversation:new`	Server to a student	Tell a student that a new chat includes them
`typing`	Client to server to others in the chat	Typing indicator
`presence`	Server to all	A student came online or went offline, with the online count
`net:probe`, `net:ping`	Acknowledged, server to client and client to server	Round-trip time measured by the server (every 5 seconds) and by the browser
`monitor:snapshot`, `activity:new`	Server to subscribed clients	Live counters every 2 seconds; new activity-log entries

Heartbeat, probes and presence. Engine.IO pings each client every 10 seconds and expects a reply within 8 seconds [4]. Separately, the server emits `net:probe` to each connection every 5 seconds and times the acknowledgement with a high-resolution clock (function `probe` in `server.js`); this is the round-trip time on the Network page. When a student's last connection closes, a `presence` event tells the others and the reason is written to the activity log.

Error handling. The server answers each kind of failure in a defined way:

- Not signed in or from another website: HTTP 401 and the Socket.IO error `unauthorized`; HTTP 403 on the API and a refused handshake on the live channel.
- Invalid input: a bad message, password or username gets an error text; a file of a disallowed type, or whose first bytes do not match its extension, gets HTTP 400, and an oversize file gets HTTP 413.
- Too many attempts: HTTP 429 after 30 sign-in attempts in 15 minutes; a connection that sends more than 15 messages in 10 seconds is told to wait.

- Lost connection: the browser reconnects by itself, shows “Cannot reach the server. Retrying” while offline, and loads what it missed once back (Figures B.2 and B.3).

3.4 Execution Procedure

Prerequisites. Node.js 22.13 or newer (check with `node --version`), a terminal, and the project folder `campus-connect` with `server.js`, `package.json`, `src/`, `public/`, `tests/` and `scripts/`. Internet access is needed once, for `npm install`. Optional environment variables are `PORT`, `DATA_DIR`, `MAX_UPLOAD_MB` and `SESSION_SECRET`.

1. Open a terminal in the project folder: `cd campus-connect`
2. Install the libraries: `npm install`
3. Start the server: `npm start`. The terminal prints `Campus Connect 1.0.0 listening on 0.0.0.0:3000`.
4. Open `http://localhost:3000` in a browser and register a student. Open a second, private browser window and register another student; each window keeps its own session cookie.
5. In the first window, start a chat with the second student, send a message and attach a file. The second window shows the message and a Download button; the Network page shows live connections, bytes and round-trip time.
6. Run the 70 automated checks with `npm test` and, with the server running, the speed test with `node scripts/bench.js http://127.0.0.1:3000`.

Figure 3.2 was taken after steps 4 and 5 with two test accounts (Asha Rao and Bilal Khan). The badge at the top right reads “Connected · WebSocket 2 ms”, and the thread shows two messages and a file card for `notes.pdf` (39.1 KB) with a Download button.

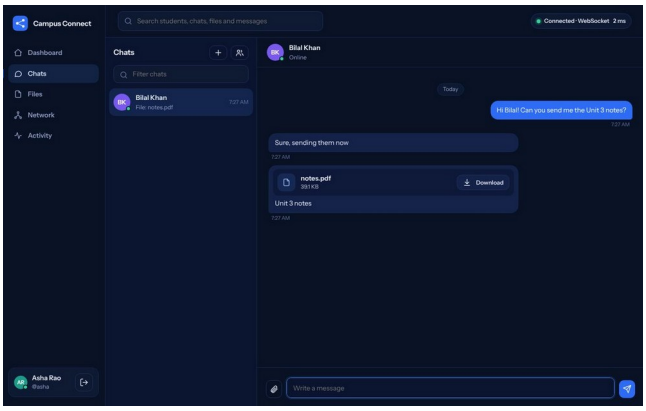


Figure 3.2 Implementation evidence: a chat with a shared file

4 Testing and Results

4.1 Test Plan

Test environment. Everything ran on the Linux computer described in Table 2.1 with Node.js 22.22.0. `npm test` starts the real server on a free port with an empty data folder, then talks to it over HTTP and Socket.IO exactly as a browser does. For the tests the upload limit is lowered to 1 MB (the default is 20 MB), and the sign-in limit is raised to 1,000 except in the last test, which sets it to 5. The clients are Node.js scripts (`fetch` and socket.io-client); headless Chromium was used for screenshots.

Success criteria. Every check must pass and the run must end with exit code 0. The 70 checks fall into nine groups, which Table 4.1 summarises as test cases T1 to T16. The test cases include normal use and also invalid input, refused access, fallback and restart.

Table 4.1 Test plan and results

Test ID	Test and Input	Expected Result	Observed Result	Status
T1	`GET /health` without signing in	HTTP 200, status ok, database ok, security headers set	As expected (Content-Security-Policy and nosniff present)	Pass
T2	Chat API and file download without sign-in; Socket.IO connection without a cookie	HTTP 401 for both; socket refused	HTTP 401 for both; socket error `unauthorized`	Pass
T3	Sign-in request with `Origin: https://evil.example`	HTTP 403	HTTP 403	Pass
T4	Register a valid student; same username in capitals; password “123”; username “no spaces!”	201; 409; 400; 400; no plain password stored	As expected; the database file holds no plain password	Pass
T5	Wrong password, then the right one; ask for the current student	401; 200; correct student returned	As expected	Pass
T6	Student A sends a message to Student B over WebSocket	Acknowledged; B receives `message:new`; unread count 1, cleared after reading	As expected	Pass
T7	Empty message; 2,001 characters; a student outside the chat posts or reads	All refused (HTTP 404 for history)	As expected	Pass
T8	Group chat of three; one member sends	Every member receives it	Both other members received it	Pass
T9	Upload a PDF; the recipient downloads it	201; live notice with name and size; identical bytes	As expected; bytes identical	Pass
T10	Text renamed `.pdf`; `.exe`; 1.2 MB file with a 1 MB limit; outsider uploads or downloads; name `../../etc/passwd.txt`	400; 400; 413; 404; name cleaned	As expected; stored name `passwd.txt`; stored files have random names	Pass
T11	Connect with long-polling only and send a message; cut the WebSocket connection	Message delivered; client reconnects by itself	Delivered over polling; reconnected	Pass

Test ID	Test and Input	Expected Result	Observed Result	Status
T12	Stop the server and start it on the same data folder	Sign-in, messages and file still there; logout ends the session	As expected	Pass
T13	Eight quick wrong sign-ins with the limit set to 5	HTTP 429	The last attempt got 429	Pass
T14	Search for a student, a file name and message text; search as an outsider	Matches found; no files from other chats	As expected	Pass
T15	Network monitor after 3 connections, 2 messages, 3 uploads, 1 download	Counters match; round-trip time and transports reported; disconnects logged	All counters matched; round-trip time measured by the server	Pass
T16	Speed test: 200 HTTP requests, 100 WebSocket and 40 polling messages, group of 10, files of 1, 5 and 15 MB	Every request completes; downloaded files identical	All completed; every download matched byte for byte	Pass

4.2 Results and Execution Evidence

Test run. Figure 4.1 shows the end of the run and selected groups; the complete output is in Appendix B. The run ended with “70 checks passed, 0 failed”. The groups shown prove two things: protection works (visitors and other websites are refused), and the fallback and reconnection work. The restart group (T12) is in the full output.

npm test, Campus Connect (21 Oct 2026, Node.js 22.22.0)

```
$ npm test
Campus Connect tests (server on port 42753)
Health and protection
ok  health endpoint answers ok
ok  security headers are set
ok  chat API needs a login
ok  file download needs a login
ok  live channel refuses visitors who are not signed in
ok  requests from another website are blocked
  ... (other groups: see Appendix B)
Polling fallback and reconnection
ok  HTTP long-polling works when WebSocket is not used
ok  messages flow over the fallback too
ok  a dropped connection reconnects by itself
Sign-in rate limit
ok  too many sign-in attempts get a 429
70 checks passed, 0 failed
```

Figure 4.1 Test execution result (selected groups of npm test)

Protocol evidence. No packet-capture tool was installed in the test environment, so the protocol steps were reproduced at application level instead. Figure 4.2 shows the replies from the running server (long lines are wrapped). The first reply is the Socket.IO polling handshake: it lists `websocket` as the allowed upgrade and reports `pingInterval` 10000, `pingTimeout` 8000 and `maxPayload` 100000, the values set in `server.js` [4], [6]. The second reply answers

a hand-written WebSocket upgrade sent over a plain TCP socket: `101 Switching Protocols` with a `Sec-WebSocket-Accept` value. That value was checked separately by taking the SHA-1 hash of the key plus the fixed GUID from RFC 6455 and encoding it in base64 [3]; it matched. The letter before the first brace is the length byte (0x69 = 105) of the first WebSocket frame, which holds the Engine.IO open packet. The last two replies show refusals: a key of the wrong length is answered with 400, and a handshake that claims to come from another website is answered with 400 and “Forbidden”.

Protocol evidence, server on 127.0.0.1:3000

```
$ curl -i "http://127.0.0.1:3000/socket.io/?EIO=4&transport=polling"
HTTP/1.1 200 OK
0{"sid":"rEXVZAjBP_J2YEKOAAAA","upgrades":["websocket"],
  "pingInterval":10000,"pingTimeout":8000,"maxPayload":100000}
# Hand-written WebSocket upgrade sent over a plain TCP socket (reply)
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: Q7UzCs0xuXibKEFzIXstDE8G03I=
i0{"sid":"y0LHMPteMYtzlcnxAAAB","upgrades":[],
  "pingInterval":10000,"pingTimeout":8000,"maxPayload":100000}
# Wrong key length, then Origin: http://evil.example (status line and body only)
HTTP/1.1 400 Bad Request
Missing or invalid Sec-WebSocket-Key header
HTTP/1.1 400 Bad Request
Forbidden
```

Figure 4.2 Protocol evidence from the running server

The same run listed the TCP sockets of the server (Table B.1 in Appendix B). It shows one socket in the LISTEN state on `0.0.0.0:3000` and, while the hand-made client was connected, an ESTABLISHED pair between `127.0.0.1:41390` and `127.0.0.1:3000`. This matches the addressing in Table 2.2: the client’s port is chosen by the operating system and the server’s port is fixed.

Network monitor. Figure 4.3 is the Network page after a short session with two students. It reports 2 active connections (2 WebSocket, 0 long-polling), 2 messages sent with 4 copies delivered, 2 files uploaded and 1 downloaded, 78.2 KB uploaded and 39.1 KB downloaded, and 49 HTTP requests with no server errors. The latest round-trip time was 1.4 ms (average 2.2 ms, minimum 1.3 ms, maximum 4.1 ms). These numbers are measured by the server with the probe in Section 3.2; they agree with what the session did (two uploads of the 39.1 KB file and one download). The Activity page recorded each connection first “over polling” and then “upgraded from polling to websocket” (Figure B.1), which is the sequence described in the Socket.IO documentation [4].

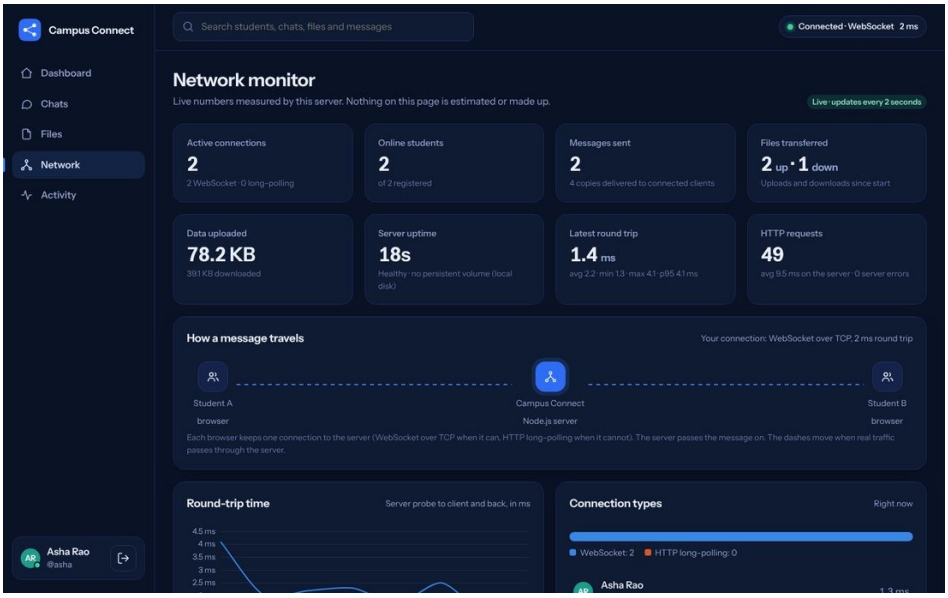


Figure 4.3 Network monitor page with live measurements

Speed test. Table 4.2 and Table 4.3 give the results of `scripts/bench.js` (T16), run on 21 October 2026. Times are measured with a high-resolution clock in one Node.js process. “Acknowledgement” is the time from sending a message until the server’s reply; “delivery” is the time from sending until the other client receives it. The full statistics are in Table B.2.

Table 4.2 Message and request timings on the loopback interface (milliseconds)

Measurement	Samples	Median	95th percentile	Maximum
HTTP `GET /health`	200	1.12	3.12	18.51
WebSocket, acknowledgement	100	1.74	3.36	10.85
WebSocket, delivery to receiver	100	1.65	2.63	11.02
Long-polling, acknowledgement	40	7.42	11.74	13.63
Long-polling, delivery to receiver	40	4.90	8.06	8.53
Group of 10 receivers, last receiver	20	2.00	14.74	14.74

Table 4.3 File transfer timings on the loopback interface (median of 3 runs, milliseconds)

File size	Upload	Download	Downloaded file identical
1 MB	14.52	5.15	Yes, byte for byte
5 MB	42.13	13.90	Yes, byte for byte
15 MB	91.57	76.24	Yes, byte for byte

Figure 4.4 compares the two transports. Because client and server share one computer, these figures contain no real network delay. They show the cost of the protocol machinery and of the application, not the speed a student would see over Wi-Fi.



Figure 4.4 Message delivery time: WebSocket compared with HTTP long-polling

4.3 Objective Achievement

Table 4.4 Objective achievement

Objective ID	Evidence or Test ID	Outcome
O1	T1, T12; Table 2.2; Figure 4.2 (server answers on port 3000)	Achieved on the test computer
O2	T6, T7, T8; Table 4.2; Figure 3.2	Achieved
O3	T9, T10, T16; Table 4.3	Achieved for files up to 15 MB. The refusal above the limit was tested with the limit set to 1 MB, not at the 20 MB default
O4	T11; Table 4.2; Figures B.2 and B.3	Achieved in the tests; behaviour behind a real proxy or firewall was not tested
O5	T2–T5, T7, T10, T13; Figure 4.2	Achieved for the cases tested; no penetration test was done
O6	T15, T16; Figures 4.3 and 4.4	Achieved, on the loopback interface only
O7	Hosting attempt (Section 5.4)	Not achieved: the hosting service paused deploys, so no public address exists

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

The working application meets six of its seven objectives; the seventh, public hosting, was not reached. The most useful result for a networking course is the comparison between the two transports. HTTP long-polling is the older approach to getting live messages from an HTTP server, and WebSocket is the approach the application prefers. Table 5.1 compares them on the same machine with the same messages, using only measured values from Table 4.2.

Table 5.1 WebSocket compared with HTTP long-polling (measured, milliseconds)

Criterion	WebSocket	Long-polling	Long-polling ÷ WebSocket
Median delivery to receiver	1.65	4.90	3.0 times
95th percentile delivery	2.63	8.06	3.1 times
Median acknowledgement	1.74	7.42	4.3 times
95th percentile acknowledgement	3.36	11.74	3.5 times

Over WebSocket a message reached the other student in a median of 1.65 ms, against 4.90 ms over long-polling, about three times longer. The likely reason is that each long-polling message needs its own HTTP request and response with headers, while an upgraded WebSocket connection only sends a small frame. That reason follows from how the protocols work [3], [4]; it was not separately profiled. The gap shows in the tail as well: the 95th percentile was 2.63 ms against 8.06 ms. A group message reached all ten receivers in a median of 2.00 ms. In the file tests, upload was slower than download at each size (for example 15 MB: 91.57 ms against 76.24 ms); the upload path also parses the multipart body, checks the first bytes and writes the file to disk.

The comparison has limits. The sample sizes are modest (100 WebSocket and 40 polling messages, 3 runs per file size), the test ran once, and loopback has no real latency, loss or bandwidth limit. The ratios describe the protocol overhead on one machine and should not be quoted as Internet performance.

5.2 Advantages and Applications

The tests demonstrate several benefits. Messages are delivered in about two milliseconds on the test machine, and the sender knows through the acknowledgement that the server accepted the message. The chat keeps working over long-polling and reconnects by itself after a break. Files reach only the people in the chat and arrive unchanged. Refused requests are refused in a

predictable way. The built-in monitor lets a student see connections, transports, bytes and round-trip time as they happen, which makes the application useful for teaching.

The same pattern, with HTTP for requests and files and WebSocket for live updates, suits small campus services: class and lab group chats, sharing notes and slides, announcements from a department, and live status pages for lab equipment or event queues.

5.3 Limitations

- Every result comes from one computer and the loopback interface. Real LAN, Wi-Fi and Internet conditions, including proxies that block WebSocket, were not tested.
- The system was not deployed. HTTPS, the `Secure` cookie flag and `wss://` have therefore not been checked on a public address.
- The tests used three students at most and a ten-receiver group. No load test with many users was run.
- One server process holds presence, counters and Socket.IO rooms in memory. Two copies of the server would need a shared adapter.
- `node:sqlite` was still marked experimental when it left its flag in Node.js 22.13 [5].
- Messages and files are stored unencrypted on the server’s disk. Uploads are checked by type and first bytes but are not scanned for malware.
- Only Chromium was used for the browser checks. The protocol evidence is from application-level tools, not from a packet capture.

5.4 Challenges and Troubleshooting

Table 5.2 records the real problems met during the work, how each was diagnosed and what was done.

Table 5.2 Challenges and troubleshooting

Issue	Cause or Diagnosis	Solution and Verification
File names with accents or other scripts came out garbled	The upload library reads file names as Latin-1	The server converts the name back to UTF-8 (`cleanName` in `src/files.js`). The check “names with accents and other scripts are kept” passes with a file named Étude 笔记.zip
A failed request appeared in the browser before sign-in	`/api/me` answered visitors with an error status	It now answers 200 with no user for visitors. The check “logout ends the session” expects exactly that reply
The speed test could be refused by the server	The server limits one connection to 15 messages per 10 seconds	The test sends one message every 0.7 seconds instead of bypassing the rule; all 100 WebSocket messages were acknowledged

Issue	Cause or Diagnosis	Solution and Verification
A hand-written WebSocket upgrade was answered with 400	The key was 17 bytes long; RFC 6455 requires a 16-byte nonce [3]	Re-sent with a 16-byte key: the server answered 101, and the accept value was recomputed and matched (Figure 4.2). The 400 reply is kept as an invalid-input case
No packet-capture tool in the test environment	No packet-capture program (such as tcpdump) was installed	Used curl, a raw TCP script and the kernel's socket table (`/proc/net/tcp`) for the evidence in Figure 4.2 and Table B.1
The public deployment failed	The hosting service answered "Deploys have been paused temporarily" on a limited trial account. The helper script `deploy.cmd` had also been blocked by Windows Smart App Control on the student's computer	Not solved. The application is run on localhost (Section 3.4) and objective O7 is reported as not achieved (Table 4.4); this report makes no claim about a live site

5.5 Future Scope

- Deploy to a host with a persistent volume, so data survives redeploys [7], and repeat the tests and the speed test from phones and laptops on different networks to measure real latency.
- Check HTTPS, the `Secure` cookie flag and `wss://` on the public address.
- Capture traffic with a packet analyser on a real LAN to show the handshake and frames at packet level.
- Add a shared adapter, such as Redis, so several server copies can share rooms and presence.
- Run a load test with many simultaneous users and large groups.
- Add password reset through the college email, malware scanning of uploads and push notifications for offline students.

5.6 Conclusion

Campus Connect is a working client–server application that uses HTTP for sign-in, history and files, and WebSocket for live messages, with a tested fallback to long-polling. All 70 automated checks passed, covering normal use, invalid input, refused access, fallback, reconnection and restart, and the files returned were identical to those sent. On one computer, a message was delivered in a median of 1.65 ms over WebSocket and 4.90 ms over long-polling. Six of the seven objectives were achieved. Public hosting was not achieved because the hosting service paused deploys, so the work has been tested on localhost only and its behaviour on a real network remains to be measured.

References

[1] R. Fielding, M. Nottingham, and J. Reschke, Eds., “HTTP Semantics,” RFC 9110, Internet Engineering Task Force, 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110 (accessed Oct. 21, 2026).

[2] W. Eddy, Ed., “Transmission Control Protocol (TCP),” RFC 9293, Internet Engineering Task Force, 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9293 (accessed Oct. 21, 2026).

[3] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, Internet Engineering Task Force, Dec. 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6455 (accessed Oct. 21, 2026).

[4] Socket.IO, “How it works,” Socket.IO Documentation, v4. [Online]. Available: https://socket.io/docs/v4/how-it-works/ (accessed Oct. 21, 2026).

[5] Node.js, “SQLite,” Node.js API Documentation. [Online]. Available: https://nodejs.org/api/sqlite.html (accessed Oct. 21, 2026).

[6] Socket.IO, “Server options,” Socket.IO Documentation, v4. [Online]. Available: https://socket.io/docs/v4/server-options/ (accessed Oct. 21, 2026).

[7] Railway, “Volumes,” Railway Docs. [Online]. Available: https://docs.railway.com/reference/volumes (accessed Oct. 21, 2026).

AI Usage Declaration

AI tools were used. Claude, an AI assistant from Anthropic, was used for code assistance, test writing, diagrams and drafting this report: it helped write the Campus Connect source code, the automated checks and the speed-test script, drew the diagrams, and drafted and edited the text from the results.

The assistant also ran `npm test`, the speed test and the protocol commands in its own Linux workspace on 21 October 2026. The outputs in Figures 4.1 and 4.2 and the numbers in Tables 4.2 and 4.3 are copied from those runs, and the screenshots come from automated browser sessions in the same workspace, so these results were obtained by running the code and not generated as text. They come from one machine, use test accounts, and have not been repeated on the student's own computer.

Checking: the code is exercised by the 70 automated checks. The standards and documentation in the References were opened and read on 21 October 2026, and the statements attributed to them were taken from those pages. The accept value in Figure 4.2 was recomputed independently. The deployment result is reported as it happened.

Declaration: We take responsibility for the accuracy of this report and understand the submitted implementation and results. Sources and assistance used have been acknowledged.

Student Name	PRN	Signature
Harsh Patil	Not shown	

Appendices

Appendix A Code and Configuration

The server code, the supporting modules and the configuration are listed below in full, with their file names. They contain no passwords or keys: the session secret is read from the `SESSION_SECRET` environment variable or generated at first start and kept in the data folder. The browser-side files in `public/`, the automated checks in `tests/run.js` and the speed test in `scripts/bench.js` are part of the submitted project folder and are not repeated here.

package.json *project description and libraries*

```
{
  "name": "campus-connect",
  "version": "1.0.0",
  "description": "Campus Connect: campus chat and file sharing (Computer Networks project)",
  "private": true,
  "main": "server.js",
  "scripts": {
    "start": "node --disable-warning=ExperimentalWarning server.js",
    "test": "node tests/run.js"
  },
  "engines": {
    "node": ">=22.13"
  },
  "dependencies": {
    "@fontsource-variable/instrument-sans": "^5.3.0",
    "bcryptjs": "^3.0.3",
    "chart.js": "^4.5.1",
    "cookie": "^2.0.1",
    "express": "^5.3.0",
    "express-rate-limit": "^8.7.1",
    "helmet": "^8.3.0",
    "multer": "^2.4.0",
    "socket.io": "^4.8.4"
  },
  "devDependencies": {
    "socket.io-client": "^4.8.4"
  }
}
```

railway.json *hosting settings (start command and health check)*

```
{
  "$schema": "https://railway.com/railway.schema.json",
  "deploy": {
    "startCommand": "npm start",
    "healthcheckPath": "/health",
    "healthcheckTimeout": 60,
    "restartPolicyType": "ON_FAILURE",
    "restartPolicyMaxRetries": 5
  }
}
```

server.js *HTTP routes, live channel and start-up*

```
'use strict';
// Campus Connect: one Node.js program that serves the website, the API, file transfers and the
live (Socket.IO) channel.
const http = require('http');
const path = require('path');
const fs = require('fs');
const crypto = require('crypto');
const express = require('express');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const multer = require('multer');
const { Server } = require('socket.io');

const { q, tx, UPLOAD_DIR } = require('./src/db');
const auth = require('./src/auth');
const files = require('./src/files');
const metrics = require('./src/metrics');
```

```
const PORT = Number(process.env.PORT) || 3000;
const MAX_MB = Number(process.env.MAX_UPLOAD_MB) || 20;
const MAX_BYTES = MAX_MB * 1024 * 1024;
const VERSION = require('./package.json').version;

const app = express();
const server = http.createServer(app);
app.set('trust proxy', 1); // Railway puts a proxy in front of the app
app.disable('x-powered-by');

const sameHost = (origin, host) => { try { return new URL(origin).host === host; } catch (_)
{ return false; } };

const io = new Server(server, {
  pingInterval: 10000,
  pingTimeout: 8000,
  maxHttpBufferSize: 1e5,
  allowRequest: (req, cb) => cb(null, !req.headers.origin || sameHost(req.headers.origin,
req.headers.host)),
});

// ---------- small helpers ----------
const presence = new Map(); // userId -> Set of socket ids (a student can have several tabs open)
const isOnline = (id) => presence.has(id);
const fmtBytes = (n) => (n < 1024 ? `${n} B` : n < 1048576 ? `${(n / 1024).toFixed(1)} KB` : `${(n
/ 1048576).toFixed(1)} MB`);

function userView(r) {
  return { id: r.id, username: r.username, displayName: r.displayName, online: isOnline(r.id),
lastSeen: r.lastSeen || null };
}
const USER_COLS = 'id, username, display_name AS displayName, last_seen AS lastSeen';
const totalStudents = () => q.get('SELECT COUNT(*) AS n FROM users').n;

function isMember(cid, uid) { return !!q.get('SELECT 1 AS x FROM members WHERE conversation_id = ?
AND user_id = ?', cid, uid); }

function messageView(id) {
  const m = q.get(
    `SELECT m.id, m.conversation_id AS conversationId, m.sender_id AS senderId, u.display_name AS
senderName, u.username AS senderUsername,
            m.body, m.created_at AS createdAt, f.id AS fileId, f.original_name AS fileName, f.size
AS fileSize, f.mime AS fileMime
       FROM messages m JOIN users u ON u.id = m.sender_id LEFT JOIN files f ON f.id = m.file_id
WHERE m.id = ?`, id);
  if (!m) return null;
  return {
    id: m.id, conversationId: m.conversationId, body: m.body, createdAt: m.createdAt,
    sender: { id: m.senderId, displayName: m.senderName, username: m.senderUsername },
    file: m.fileId ? { id: m.fileId, name: m.fileName, size: m.fileSize, mime: m.fileMime,
isImage: files.IMAGE_MIMES.has(m.fileMime) } : null,
  };
}

function conversationView(cid, myId) {
  const c = q.get('SELECT id, kind, name, created_at AS createdAt FROM conversations WHERE id
= ?', cid);
  if (!c) return null;
  const members = q.all(
    `SELECT u.id, u.username, u.display_name AS displayName, u.last_seen AS lastSeen FROM members
m JOIN users u ON u.id = m.user_id
     WHERE m.conversation_id = ? ORDER BY u.display_name`, cid).map(userView);
  const peer = c.kind === 'direct' ? members.find((m) => m.id !== myId) || members[0] : null;
  const last = q.get(
    `SELECT m.id, m.body, m.sender_id AS senderId, m.created_at AS createdAt, f.original_name AS
fileName
       FROM messages m LEFT JOIN files f ON f.id = m.file_id WHERE m.conversation_id = ? ORDER BY
m.id DESC LIMIT 1`, cid);
  const unread = q.get(
    `SELECT COUNT(*) AS n FROM messages WHERE conversation_id = ? AND sender_id != ?
       AND id > (SELECT last_read_id FROM members WHERE conversation_id = ? AND user_id = ?)`,
cid, myId, cid, myId).n;
  return {
    id: c.id, kind: c.kind, name: c.kind === 'direct' ? peer.displayName : c.name, peer, members,
    createdAt: c.createdAt, unread,
    last: last ? { id: last.id, text: last.fileName ? `File: ${last.fileName}` : last.body,
senderId: last.senderId, createdAt: last.createdAt } : null,
  };
}

function myConversations(uid) {
  const ids = q.all('SELECT conversation_id AS id FROM members WHERE user_id = ?', uid).map((r) =>
r.id);
```

```
  return ids.map((id) => conversationView(id, uid)).filter(Boolean)
    .sort((a, b) => ((b.last ? b.last.createdAt : b.createdAt) - (a.last ? a.last.createdAt :
a.createdAt)));
}

function addToRooms(cid, userIds) {
  for (const id of userIds) {
    io.in(`user:${id}`).socketsJoin(`conv:${cid}`);
    io.to(`user:${id}`).emit('conversation:new', { conversationId: cid });
  }
}

// Save a message, push it to everyone in the conversation, and count it.
function postMessage({ cid, userId, body, fileId = null }) {
  const id = tx(() => {
    const r = q.run('INSERT INTO messages (conversation_id, sender_id, body, file_id, created_at)
VALUES (?,?,?,?,?)', cid, userId, body, fileId, Date.now());
    const mid = Number(r.lastInsertRowid);
    q.run('UPDATE members SET last_read_id = ? WHERE conversation_id = ? AND user_id = ?', mid,
cid, userId);
    return mid;
  });
  const msg = messageView(id);
  const room = io.sockets.adapter.rooms.get(`conv:${cid}`) || new Set();
  let deliveries = 0;
  for (const sid of room) {
    const s = io.sockets.sockets.get(sid);
    if (s && s.data.user.id !== userId) deliveries += 1;
  }
  io.to(`conv:${cid}`).emit('message:new', msg);
  io.to('monitor').emit('monitor:pulse', { kind: msg.file ? 'file' : 'message' });
  return { msg, deliveries };
}

// ---------- middleware ----------
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      imgSrc: ["'self'", 'data:', 'blob:'],
      connectSrc: ["'self'", 'ws:', 'wss:'],
      fontSrc: ["'self'"],
      objectSrc: ["'none'"],
      frameAncestors: ["'none'"],
      baseUri: ["'self'"],
      formAction: ["'self'"],
      upgradeInsecureRequests: null,
    },
  },
  crossOriginEmbedderPolicy: false,
}));
app.use((req, res, next) => {
  const t0 = process.hrtime.bigint();
  res.on('finish', () => metrics.onHttp(Number(process.hrtime.bigint() - t0) / 1e6,
res.statusCode));
  next();
});
// Block state-changing requests that come from another website.
app.use((req, res, next) => {
  if (['GET', 'HEAD', 'OPTIONS'].includes(req.method)) return next();
  if (req.headers.origin && !sameHost(req.headers.origin, req.headers.host)) return
res.status(403).json({ error: 'Request blocked.' });
  next();
});
app.use(express.json({ limit: '20kb' }));

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: Number(process.env.AUTH_RATE_LIMIT) || 30,
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: 'Too many attempts. Wait a few minutes and try again.' },
});

// ---------- health ----------
app.get('/health', (req, res) => {
  try {
    q.get('SELECT 1 AS ok');
    fs.accessSync(UPLOAD_DIR, fs.constants.W_OK);
    res.json({
      status: 'ok', version: VERSION, time: new Date().toISOString(), uptimeSec:
Math.floor(process.uptime()),
```

```
      database: 'ok', storageWritable: true, persistentVolume:
Boolean(process.env.RAILWAY_VOLUME_MOUNT_PATH),
      connections: metrics.connections.size,
    });
  } catch (err) {
    res.status(503).json({ status: 'error', message: 'Storage or database problem.' });
  }
});

// ---------- accounts ----------
app.post('/api/auth/register', authLimiter, async (req, res) => {
  const { errors, name, user } = auth.validateRegistration(req.body || {});
  if (Object.keys(errors).length) return res.status(400).json({ error: 'Please fix the highlighted
fields.', errors });
  let id;
  try {
    id = await auth.createUser(name, user, req.body.password);
  } catch (err) {
    if (/UNIQUE/i.test(String(err.message))) return res.status(409).json({ error: 'That username
is already taken.', errors: { username: 'That username is already taken.' } });
    throw err;
  }
  auth.startSession(res, id);
  const me = q.get(`SELECT ${USER_COLS} FROM users WHERE id = ?`, id);
  metrics.logActivity('register', id, 'created an account');
  io.emit('user:new', { user: userView(me), totalStudents: totalStudents() });
  res.status(201).json({ user: userView(me) });
});

app.post('/api/auth/login', authLimiter, async (req, res) => {
  const id = await auth.checkLogin(req.body && req.body.username, req.body && req.body.password);
  if (!id) return res.status(401).json({ error: 'Wrong username or password.' });
  auth.startSession(res, id);
  metrics.logActivity('login', id, 'signed in');
  res.json({ user: userView(q.get(`SELECT ${USER_COLS} FROM users WHERE id = ?`, id)) });
});

app.post('/api/auth/logout', auth.requireAuth, (req, res) => {
  auth.endSession(req.sessionToken);
  metrics.logActivity('logout', req.user.id, 'signed out');
  io.in(`user:${req.user.id}`).disconnectSockets(true);
  res.clearCookie(auth.COOKIE, { path: '/' });
  res.json({ ok: true });
});

// Answers 200 for everyone so the sign-in screen does not cause a red 401 in the browser console.
// Signed-out visitors get { user: null }.
app.get('/api/me', (req, res) => {
  const config = { maxUploadMb: MAX_MB, allowedExtensions: files.allowedList() };
  const user = auth.userFromToken(auth.tokenFromCookieHeader(req.headers.cookie));
  if (!user) return res.json({ user: null, config });
  res.json({ user: userView(q.get(`SELECT ${USER_COLS} FROM users WHERE id = ?`, user.id)), config
});
});

// ---------- people and conversations ----------
app.get('/api/users', auth.requireAuth, (req, res) => {
  const rows = q.all(`SELECT ${USER_COLS} FROM users WHERE id != ? ORDER BY display_name LIMIT
500`, req.user.id);
  res.json({ users: rows.map(userView), totalStudents: totalStudents(), onlineCount: presence.size
});
});

app.get('/api/conversations', auth.requireAuth, (req, res) => {
  res.json({ conversations: myConversations(req.user.id) });
});

app.post('/api/conversations', auth.requireAuth, (req, res) => {
  const body = req.body || {};
  const me = req.user.id;
  if (body.kind === 'direct') {
    const other = Number(body.userId);
    if (!Number.isInteger(other) || other === me || !q.get('SELECT 1 AS x FROM users WHERE id
= ?', other)) return res.status(400).json({ error: 'Choose another student.' });
    const key = `${Math.min(me, other)}:${Math.max(me, other)}`;
    let c = q.get('SELECT id FROM conversations WHERE direct_key = ?', key);
    let created = false;
    if (!c) {
      const cid = tx(() => {
        const r = q.run("INSERT INTO conversations (kind, direct_key, created_by, created_at)
VALUES ('direct', ?, ?, ?)", key, me, Date.now());
        const id = Number(r.lastInsertRowid);
        for (const u of [me, other]) q.run('INSERT INTO members (conversation_id, user_id,
joined_at) VALUES (?,?,?)', id, u, Date.now());
```

```
        return id;
      });
      c = { id: cid };
      created = true;
      addToRooms(cid, [me, other]);
    }
    return res.status(created ? 201 : 200).json({ conversation: conversationView(c.id, me) });
  }
  if (body.kind === 'group') {
    const name = typeof body.name === 'string' ? body.name.trim().replace(/\s+/g, ' ') : '';
    if (name.length < 2 || name.length > 40 || /[<>\u0000-\u001f]/.test(name)) return
res.status(400).json({ error: 'Give the group a name (2 to 40 characters).' });
    const ids = [...new Set((Array.isArray(body.memberIds) ? body.memberIds : []).map(Number))]
      .filter((n) => Number.isInteger(n) && n !== me && q.get('SELECT 1 AS x FROM users WHERE id =
?', n)).slice(0, 49);
    if (!ids.length) return res.status(400).json({ error: 'Add at least one other student to the
group.' });
    const cid = tx(() => {
      const r = q.run("INSERT INTO conversations (kind, name, created_by, created_at) VALUES
('group', ?, ?, ?)", name, me, Date.now());
      const id = Number(r.lastInsertRowid);
      for (const u of [me, ...ids]) q.run('INSERT INTO members (conversation_id, user_id,
joined_at) VALUES (?,?,?)', id, u, Date.now());
      return id;
    });
    addToRooms(cid, [me, ...ids]);
    return res.status(201).json({ conversation: conversationView(cid, me) });
  }
  res.status(400).json({ error: 'Unknown conversation type.' });
});

function memberOnly(req, res, next) {
  const cid = Number(req.params.id);
  if (!Number.isInteger(cid) || !isMember(cid, req.user.id)) return res.status(404).json({ error:
'Conversation not found.' });
  req.cid = cid;
  next();
}

app.get('/api/conversations/:id/messages', auth.requireAuth, memberOnly, (req, res) => {
  const limit = Math.min(Math.max(parseInt(req.query.limit, 10) || 60, 1), 200);
  const before = parseInt(req.query.before, 10);
  const rows = before
    ? q.all('SELECT id FROM messages WHERE conversation_id = ? AND id < ? ORDER BY id DESC
LIMIT ?', req.cid, before, limit)
    : q.all('SELECT id FROM messages WHERE conversation_id = ? ORDER BY id DESC LIMIT ?', req.cid,
limit);
  res.json({ messages: rows.reverse().map((r) => messageView(r.id)), hasMore: rows.length ===
limit });
});

function markRead(uid, cid, upTo) {
  const top = q.get('SELECT COALESCE(MAX(id),0) AS m FROM messages WHERE conversation_id = ?',
cid).m;
  const upto = Math.min(Number(upTo) || top, top);
  q.run('UPDATE members SET last_read_id = MAX(last_read_id, ?) WHERE conversation_id = ? AND
user_id = ?', upto, cid, uid);
  io.to(`user:${uid}`).emit('conversation:read', { conversationId: cid });
}
app.post('/api/conversations/:id/read', auth.requireAuth, memberOnly, (req, res) => {
  markRead(req.user.id, req.cid, req.body && req.body.upTo);
  res.json({ ok: true });
});

// ---------- file sharing (uploads use plain HTTP; the live channel only announces them)
----------
const upload = multer({
  storage: multer.diskStorage({
    destination: UPLOAD_DIR,
    filename: (req, file, cb) => cb(null, crypto.randomBytes(16).toString('hex')), // never use
the student's file name on disk
  }),
  limits: { fileSize: MAX_BYTES, files: 1, fields: 4, parts: 8 },
  fileFilter: (req, file, cb) => {
    if (files.typeFor(files.cleanName(file.originalname))) return cb(null, true);
    cb(Object.assign(new Error('type'), { userMessage: 'This file type is not allowed. Use a PDF,
image, Office or text document, code file, or ZIP.' }));
  },
});

function runUpload(req, res, next) {
  upload.single('file')(req, res, (err) => {
    if (!err) return next();
```

```
    if (err.code === 'LIMIT_FILE_SIZE') return res.status(413).json({ error: `That file is larger
than ${MAX_MB} MB.` });
    if (err.userMessage) return res.status(400).json({ error: err.userMessage });
    if (err instanceof multer.MulterError) return res.status(400).json({ error: 'The upload could
not be read. Try again.' });
    next(err);
  });
}

app.post('/api/conversations/:id/files', auth.requireAuth, memberOnly, runUpload, (req, res) => {
  if (!req.file) return res.status(400).json({ error: 'Choose a file to send.' });
  const stored = path.join(UPLOAD_DIR, req.file.filename);
  const name = files.cleanName(req.file.originalname);
  const [mime, kind] = files.typeFor(name);
  if (!files.contentLooksRight(stored, kind)) {
    fs.unlink(stored, () => {});
    return res.status(400).json({ error: `"${name}" does not look like a real .$
{files.extOf(name)} file.` });
  }
  const caption = typeof req.body.caption === 'string' ? req.body.caption.trim().slice(0, 500) :
'';
  const fileId = Number(q.run(
    'INSERT INTO files (conversation_id, uploader_id, original_name, stored_name, mime, size,
created_at) VALUES (?,?,?,?,?,?,?)',
    req.cid, req.user.id, name, req.file.filename, mime, req.file.size,
Date.now()).lastInsertRowid);
  const { msg, deliveries } = postMessage({ cid: req.cid, userId: req.user.id, body: caption,
fileId });
  metrics.onUpload(req.file.size);
  metrics.counters.deliveries += deliveries;
  metrics.logActivity('upload', req.user.id, `shared a file (${fmtBytes(req.file.size)})`);
  res.status(201).json({ message: msg });
});

app.get('/api/files', auth.requireAuth, (req, res) => {
  const me = req.user.id;
  const rows = q.all(
    `SELECT f.id, f.original_name AS name, f.size, f.mime, f.downloads, f.created_at AS createdAt,
f.uploader_id AS uploaderId,
            u.display_name AS uploaderName, f.conversation_id AS conversationId, c.kind, c.name AS
groupName
       FROM files f JOIN members m ON m.conversation_id = f.conversation_id AND m.user_id = ?
       JOIN users u ON u.id = f.uploader_id JOIN conversations c ON c.id = f.conversation_id
      ORDER BY f.id DESC LIMIT 100`, me);
  const out = rows.map((r) => {
    let to = r.groupName;
    if (r.kind === 'direct') {
      const peer = q.get('SELECT u.display_name AS n FROM members m JOIN users u ON u.id =
m.user_id WHERE m.conversation_id = ? AND m.user_id != ?', r.conversationId, r.uploaderId);
      to = r.uploaderId === me ? (peer ? peer.n : '') : 'You';
    }
    return { ...r, to, sentByMe: r.uploaderId === me, isImage: files.IMAGE_MIMES.has(r.mime) };
  });
  res.json({ files: out });
});

app.get('/api/files/:id', auth.requireAuth, (req, res) => {
  const f = q.get('SELECT * FROM files WHERE id = ?', Number(req.params.id));
  // Same answer for "does not exist" and "not yours", so file ids cannot be probed.
  if (!f || !isMember(f.conversation_id, req.user.id)) return res.status(404).json({ error: 'File
not found.' });
  const full = path.join(UPLOAD_DIR, f.stored_name);
  let stat;
  try { stat = fs.statSync(full); } catch (_) { return res.status(404).json({ error: 'File not
found.' }); }
  const inline = req.query.inline === '1' && files.IMAGE_MIMES.has(f.mime);
  if (inline) res.setHeader('Content-Disposition', 'inline');
  else res.attachment(f.original_name);
  res.setHeader('Content-Type', f.mime);
  res.setHeader('Content-Length', stat.size);
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('Cache-Control', 'private, no-store');
  res.setHeader('Content-Security-Policy', "default-src 'none'; sandbox");
  let sent = 0;
  const stream = fs.createReadStream(full);
  stream.on('data', (c) => { sent += c.length; });
  stream.on('error', () => res.destroy());
  res.on('close', () => {
    if (inline || sent < stat.size) return;
    metrics.onDownload(sent);
    q.run('UPDATE files SET downloads = downloads + 1 WHERE id = ?', f.id);
    metrics.logActivity('download', req.user.id, `downloaded a file (${fmtBytes(sent)})`);
  });
  stream.pipe(res);
```

```
});

// ---------- dashboard and search ----------
app.get('/api/dashboard', auth.requireAuth, (req, res) => {
  const me = req.user.id;
  const convs = myConversations(me);
  const fr = q.all(
    `SELECT f.id, f.original_name AS name, f.size, f.created_at AS createdAt, f.uploader_id AS
uploaderId, u.display_name AS uploaderName
       FROM files f JOIN members m ON m.conversation_id = f.conversation_id AND m.user_id = ?
       JOIN users u ON u.id = f.uploader_id ORDER BY f.id DESC LIMIT 5`, me);
  res.json({
    totalStudents: totalStudents(), onlineCount: presence.size,
    unread: convs.reduce((a, c) => a + c.unread, 0),
    conversations: convs.slice(0, 5), files: fr.map((f) => ({ ...f, sentByMe: f.uploaderId ===
me })),
    activity: metrics.recentActivity(8),
  });
});

app.get('/api/search', auth.requireAuth, (req, res) => {
  const term = String(req.query.q || '').trim().slice(0, 60);
  if (term.length < 2) return res.json({ users: [], conversations: [], files: [], messages: [] });
  const like = `%${term.replace(/[\\%_]/g, (m) => `\\${m}`)}%`;
  const me = req.user.id;
  const users = q.all(`SELECT ${USER_COLS} FROM users WHERE id != ? AND (display_name LIKE ?
ESCAPE '\\' OR username LIKE ? ESCAPE '\\') ORDER BY display_name LIMIT 8`, me, like,
like).map(userView);
  const convs = myConversations(me).filter((c) =>
c.name.toLowerCase().includes(term.toLowerCase())).slice(0, 6);
  const fl = q.all(
    `SELECT f.id, f.original_name AS name, f.size, f.conversation_id AS conversationId FROM files
f
       JOIN members m ON m.conversation_id = f.conversation_id AND m.user_id = ?
      WHERE f.original_name LIKE ? ESCAPE '\\' ORDER BY f.id DESC LIMIT 6`, me, like);
  const msgs = q.all(
    `SELECT m.id, m.conversation_id AS conversationId, m.body, m.created_at AS createdAt,
u.display_name AS senderName FROM messages m
       JOIN members mm ON mm.conversation_id = m.conversation_id AND mm.user_id = ? JOIN users u
ON u.id = m.sender_id
      WHERE m.body LIKE ? ESCAPE '\\' ORDER BY m.id DESC LIMIT 6`, me, like);
  res.json({ users, conversations: convs.map((c) => ({ id: c.id, name: c.name, kind: c.kind })),
files: fl, messages: msgs });
});

// ---------- network monitor ----------
function monitorSnapshot() {
  return metrics.snapshot({
    onlineStudents: presence.size,
    totalStudents: totalStudents(),
    storage: { persistentVolume: Boolean(process.env.RAILWAY_VOLUME_MOUNT_PATH), maxUploadMb:
MAX_MB },
    server: { node: process.version, version: VERSION },
  });
}
app.get('/api/monitor', auth.requireAuth, (req, res) => {
  res.json({ snapshot: monitorSnapshot(), activity: metrics.recentActivity(30) });
});
app.get('/api/activity', auth.requireAuth, (req, res) => {
  const kinds = typeof req.query.kind === 'string' && req.query.kind ?
req.query.kind.split(',').slice(0, 8) : null;
  res.json({ activity: metrics.recentActivity(100, kinds) });
});

// ---------- the website itself ----------
app.use(express.static(path.join(__dirname, 'public'), {
  index: 'index.html',
  setHeaders: (res, file) => {
    if (/\.(woff2|js|css)$/.test(file) && /\/(fonts|vendor)\//.test(file)) res.setHeader('Cache-
Control', 'public, max-age=604800, immutable');
    else res.setHeader('Cache-Control', 'no-cache');
  },
}));
app.use('/api', (req, res) => res.status(404).json({ error: 'Not found.' }));

// eslint-disable-next-line no-unused-vars
app.use((err, req, res, next) => {
  console.error('Request error:', err && err.stack ? err.stack : err);
  if (res.headersSent) return res.destroy();
  res.status(err && err.status && err.status < 500 ? err.status : 500).json({ error: 'Something
went wrong on the server.' });
});
```

```
// ---------- live channel (Socket.IO: WebSocket first choice, HTTP long-polling as fallback)
----------
io.use((socket, next) => {
  const user = auth.userFromToken(auth.tokenFromCookieHeader(socket.request.headers.cookie));
  if (!user) return next(new Error('unauthorized'));
  socket.data.user = user;
  next();
});

metrics.onActivity((row) => {
  const who = row.userId ? q.get('SELECT display_name AS name, username FROM users WHERE id = ?',
row.userId) : null;
  const evt = { ...row, name: who ? who.name : null, username: who ? who.username : null };
  io.to('feed').emit('activity:new', evt);
});

function probe(s) {
  const t0 = process.hrtime.bigint();
  s.timeout(4000).emit('net:probe', (err) => {
    if (err) return;
    const ms = Number(process.hrtime.bigint() - t0) / 1e6;
    metrics.recordRtt(s.data.user.id, ms);
    const c = metrics.connections.get(s.id);
    if (c) c.rtt = Math.round(ms * 10) / 10;
  });
}

io.on('connection', (socket) => {
  const user = socket.data.user;
  const transport = socket.conn.transport.name;
  metrics.counters.connectionsOpened += 1;
  metrics.connections.set(socket.id, { userId: user.id, name: user.displayName, username:
user.username, transport, since: Date.now(), rtt: null });
  socket.join([`user:${user.id}`, 'feed']);
  for (const r of q.all('SELECT conversation_id AS id FROM members WHERE user_id = ?', user.id))
socket.join(`conv:${r.id}`);

  const wasOnline = isOnline(user.id);
  if (!presence.has(user.id)) presence.set(user.id, new Set());
  presence.get(user.id).add(socket.id);
  if (!wasOnline) io.emit('presence', { userId: user.id, online: true, onlineCount:
presence.size });
  metrics.logActivity('connect', user.id, `connected over ${transport}`);
  socket.emit('ready', { socketId: socket.id, transport });
  setTimeout(() => { if (socket.connected) probe(socket); }, 600);

  socket.conn.on('upgrade', (t) => {
    const c = metrics.connections.get(socket.id);
    if (c) c.transport = t.name;
    metrics.logActivity('upgrade', user.id, `upgraded from ${transport} to ${t.name}`);
  });

  // client-side round-trip test: the client sends, we answer at once, the client times it
  socket.on('net:ping', (ack) => { if (typeof ack === 'function') ack(Date.now()); });

  let burst = []; // timestamps of this socket's recent messages (simple flood limit)
  socket.on('message:send', (payload, ack) => {
    const reply = typeof ack === 'function' ? ack : () => {};
    try {
      const cid = Number(payload && payload.conversationId);
      const body = typeof (payload && payload.body) === 'string' ? payload.body.trim() : '';
      if (!Number.isInteger(cid) || !isMember(cid, user.id)) return reply({ ok: false, error:
'Conversation not found.' });
      if (!body) return reply({ ok: false, error: 'Type a message first.' });
      if (body.length > 2000) return reply({ ok: false, error: 'Messages can be up to 2000
characters.' });
      const now = Date.now();
      burst = burst.filter((t) => now - t < 10000);
      if (burst.length >= 15) return reply({ ok: false, error: 'You are sending too fast. Wait a
moment.' });
      burst.push(now);
      const { msg, deliveries } = postMessage({ cid, userId: user.id, body });
      metrics.onMessage(deliveries);
      const kind = q.get('SELECT kind FROM conversations WHERE id = ?', cid).kind;
      metrics.logActivity('message', user.id, `sent a message in a ${kind === 'group' ? 'group' :
'direct'} chat`);
      reply({ ok: true, message: msg });
    } catch (err) {
      console.error('message:send failed', err);
      reply({ ok: false, error: 'The message could not be saved.' });
    }
  });

  socket.on('conversation:read', (p) => {
```

```
    const cid = Number(p && p.conversationId);
    if (Number.isInteger(cid) && isMember(cid, user.id)) markRead(user.id, cid, p.upTo);
  });
  socket.on('typing', (p) => {
    const cid = Number(p && p.conversationId);
    if (Number.isInteger(cid) && isMember(cid, user.id)) socket.to(`conv:${cid}`).emit('typing', {
conversationId: cid, userId: user.id, name: user.displayName, typing: Boolean(p.typing) });
  });
  socket.on('monitor:subscribe', () => { socket.join('monitor'); socket.emit('monitor:snapshot',
monitorSnapshot()); });
  socket.on('monitor:unsubscribe', () => socket.leave('monitor'));

  socket.on('disconnect', (reason) => {
    metrics.counters.disconnections += 1;
    metrics.connections.delete(socket.id);
    const set = presence.get(user.id);
    if (set) {
      set.delete(socket.id);
      if (!set.size) {
        presence.delete(user.id);
        q.run('UPDATE users SET last_seen = ? WHERE id = ?', Date.now(), user.id);
        io.emit('presence', { userId: user.id, online: false, onlineCount: presence.size,
lastSeen: Date.now() });
      }
    }
    metrics.logActivity('disconnect', user.id, `disconnected (${reason})`);
  });
});

setInterval(() => { for (const s of io.sockets.sockets.values()) probe(s); }, 5000).unref();
setInterval(() => {
  const room = io.sockets.adapter.rooms.get('monitor');
  if (room && room.size) io.to('monitor').emit('monitor:snapshot', monitorSnapshot());
}, 2000).unref();

server.listen(PORT, '0.0.0.0', () => console.log(`Campus Connect ${VERSION} listening on 0.0.0.0:$
{PORT}`));

function shutdown(signal) {
  console.log(`${signal} received, shutting down`);
  io.close(() => process.exit(0));
  setTimeout(() => process.exit(0), 4000).unref();
}
process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT', () => shutdown('SIGINT'));
```

src/auth.js *passwords, sessions and sign-in checks*

```
'use strict';
// Accounts, passwords and login sessions.
const crypto = require('crypto');
const fs = require('fs');
const path = require('path');
const bcrypt = require('bcryptjs');
const { parseCookie } = require('cookie');
const { q, DATA_DIR } = require('./db');

const COOKIE = 'cc_session';
const SESSION_MS = 7 * 24 * 60 * 60 * 1000;
const IS_PROD = process.env.NODE_ENV === 'production';

// The secret comes from the SESSION_SECRET variable. If it is missing, make one and keep it in
the data folder.
function loadSecret() {
  if (process.env.SESSION_SECRET && process.env.SESSION_SECRET.length >= 16) return
process.env.SESSION_SECRET;
  const file = path.join(DATA_DIR, '.session_secret');
  try {
    return fs.readFileSync(file, 'utf8').trim();
  } catch (_) {
    const s = crypto.randomBytes(32).toString('hex');
    fs.writeFileSync(file, s, { mode: 0o600 });
    return s;
  }
}
const SECRET = loadSecret();

const hashToken = (token) => crypto.createHmac('sha256', SECRET).update(token).digest('hex');
const DUMMY_HASH = bcrypt.hashSync('not-a-real-password', 10); // makes "unknown user" take as
long as "wrong password"

function validateRegistration({ displayName, username, password }) {
  const errors = {};
```

```
  const name = typeof displayName === 'string' ? displayName.trim().replace(/\s+/g, ' ') : '';
  const user = typeof username === 'string' ? username.trim().toLowerCase() : '';
  if (name.length < 2 || name.length > 50) errors.displayName = 'Enter your name (2 to 50
characters).';
  else if (/[<>\u0000-\u001f]/.test(name)) errors.displayName = 'Your name contains characters
that are not allowed.';
  if (!/^[a-z0-9_.]{3,24}$/.test(user)) errors.username = 'Username: 3 to 24 letters, numbers,
dots or underscores.';
  if (typeof password !== 'string' || password.length < 8) errors.password = 'Password must be at
least 8 characters.';
  else if (Buffer.byteLength(password) > 72) errors.password = 'Password is too long (72
characters at most).';
  return { errors, name, user };
}

async function createUser(name, username, password) {
  const hash = await bcrypt.hash(password, 10);
  const r = q.run('INSERT INTO users (username, display_name, password_hash, created_at) VALUES
(?,?,?,?)', username, name, hash, Date.now());
  return Number(r.lastInsertRowid);
}

async function checkLogin(username, password) {
  const u = q.get('SELECT id, password_hash FROM users WHERE username = ?', String(username ||
'').trim().toLowerCase());
  const ok = await bcrypt.compare(String(password || ''), u ? u.password_hash : DUMMY_HASH);
  return ok && u ? u.id : null;
}

function startSession(res, userId) {
  const token = crypto.randomBytes(32).toString('base64url');
  const now = Date.now();
  q.run('INSERT INTO sessions (token_hash, user_id, created_at, expires_at) VALUES (?,?,?,?)',
hashToken(token), userId, now, now + SESSION_MS);
  q.run('DELETE FROM sessions WHERE expires_at < ?', now);
  res.cookie(COOKIE, token, { httpOnly: true, sameSite: 'lax', secure: IS_PROD, maxAge:
SESSION_MS, path: '/' });
}

function tokenFromCookieHeader(header) {
  if (!header) return null;
  return parseCookie(header)[COOKIE] || null;
}

function userFromToken(token) {
  if (!token) return null;
  const row = q.get(
    `SELECT u.id, u.username, u.display_name AS displayName, s.expires_at AS exp
       FROM sessions s JOIN users u ON u.id = s.user_id WHERE s.token_hash = ?`,
hashToken(token));
  if (!row || row.exp < Date.now()) return null;
  return { id: row.id, username: row.username, displayName: row.displayName };
}

function endSession(token) {
  if (token) q.run('DELETE FROM sessions WHERE token_hash = ?', hashToken(token));
}

// Express middleware: only logged-in users get through.
function requireAuth(req, res, next) {
  const token = tokenFromCookieHeader(req.headers.cookie);
  const user = userFromToken(token);
  if (!user) return res.status(401).json({ error: 'Please sign in.' });
  req.user = user;
  req.sessionToken = token;
  next();
}

module.exports = {
  COOKIE, validateRegistration, createUser, checkLogin, startSession, endSession,
  tokenFromCookieHeader, userFromToken, requireAuth,
};
```

src/files.js *allowed file types, name cleaning and content check*

```
'use strict';
// Which files are allowed, how names are cleaned, and a basic content check.
const fs = require('fs');
const path = require('path');

// extension -> [mime type we send back, content check]
const PDF = 'pdf', PNG = 'png', JPG = 'jpg', GIF = 'gif', WEBP = 'webp', ZIP = 'zip', OLE = 'ole',
SZ = '7z', RTF = 'rtf', TEXT = 'text';
```

```
const ALLOWED = {
  pdf: ['application/pdf', PDF],
  png: ['image/png', PNG], jpg: ['image/jpeg', JPG], jpeg: ['image/jpeg', JPG], gif: ['image/gif',
GIF], webp: ['image/webp', WEBP],
  zip: ['application/zip', ZIP], '7z': ['application/x-7z-compressed', SZ],
  docx: ['application/vnd.openxmlformats-officedocument.wordprocessingml.document', ZIP],
  xlsx: ['application/vnd.openxmlformats-officedocument.spreadsheetml.sheet', ZIP],
  pptx: ['application/vnd.openxmlformats-officedocument.presentationml.presentation', ZIP],
  odt: ['application/vnd.oasis.opendocument.text', ZIP], ods:
['application/vnd.oasis.opendocument.spreadsheet', ZIP],
  odp: ['application/vnd.oasis.opendocument.presentation', ZIP],
  doc: ['application/msword', OLE], xls: ['application/vnd.ms-excel', OLE], ppt:
['application/vnd.ms-powerpoint', OLE],
  rtf: ['application/rtf', RTF],
  txt: ['text/plain; charset=utf-8', TEXT], md: ['text/plain; charset=utf-8', TEXT], csv:
['text/csv; charset=utf-8', TEXT],
  c: ['text/plain; charset=utf-8', TEXT], cpp: ['text/plain; charset=utf-8', TEXT], h:
['text/plain; charset=utf-8', TEXT],
  java: ['text/plain; charset=utf-8', TEXT], py: ['text/plain; charset=utf-8', TEXT], js:
['text/plain; charset=utf-8', TEXT],
  sql: ['text/plain; charset=utf-8', TEXT], json: ['text/plain; charset=utf-8', TEXT],
};
const IMAGE_MIMES = new Set(['image/png', 'image/jpeg', 'image/gif', 'image/webp']);

function extOf(name) { return path.extname(name).slice(1).toLowerCase(); }
function typeFor(name) { return ALLOWED[extOf(name)] || null; }
function allowedList() { return Object.keys(ALLOWED).sort(); }

// Multer reads names as latin1; turn them back into proper UTF-8, then remove anything risky.
function cleanName(raw) {
  let n = Buffer.from(String(raw || ''), 'latin1').toString('utf8');
  n = path.basename(n.replace(/\\/g, '/')).normalize('NFC');
  n = n.replace(/[\u0000-\u001f\u007f<>:"|?*\/\\]/g, '_').replace(/^\.+/, '').trim();
  if (!n) n = 'file';
  if (n.length > 120) {
    const ext = path.extname(n).slice(0, 12);
    n = n.slice(0, 120 - ext.length) + ext;
  }
  return n;
}

function startsWith(buf, bytes) { return bytes.every((b, i) => buf[i] === b); }

// Read the first bytes of the saved file and check that they match what the extension says.
function contentLooksRight(file, kind) {
  const fd = fs.openSync(file, 'r');
  const buf = Buffer.alloc(4096);
  const n = fs.readSync(fd, buf, 0, 4096, 0);
  fs.closeSync(fd);
  const b = buf.subarray(0, n);
  if (n === 0) return false;
  switch (kind) {
    case PDF: return b.subarray(0, 1024).includes('%PDF-');
    case PNG: return startsWith(b, [0x89, 0x50, 0x4e, 0x47]);
    case JPG: return startsWith(b, [0xff, 0xd8, 0xff]);
    case GIF: return startsWith(b, [0x47, 0x49, 0x46, 0x38]);
    case WEBP: return b.subarray(0, 4).toString() === 'RIFF' && b.subarray(8, 12).toString() ===
'WEBP';
    case ZIP: return startsWith(b, [0x50, 0x4b, 0x03, 0x04]) || startsWith(b, [0x50, 0x4b, 0x05,
0x06]);
    case SZ: return startsWith(b, [0x37, 0x7a, 0xbc, 0xaf, 0x27, 0x1c]);
    case OLE: return startsWith(b, [0xd0, 0xcf, 0x11, 0xe0, 0xa1, 0xb1, 0x1a, 0xe1]);
    case RTF: return b.subarray(0, 5).toString() === '{\\rtf';
    case TEXT: return !b.includes(0);
    default: return false;
  }
}

module.exports = { typeFor, allowedList, cleanName, contentLooksRight, extOf, IMAGE_MIMES };
```

src/db.js *database set-up*

```
'use strict';
// Database and storage. SQLite (built into Node 22) + a data folder for uploaded files.
const fs = require('fs');
const path = require('path');
const { DatabaseSync } = require('node:sqlite');

// On Railway the persistent volume is mounted at /app/data (Railway also sets
RAILWAY_VOLUME_MOUNT_PATH).
function resolveDataDir() {
  const wanted = [process.env.DATA_DIR, process.env.RAILWAY_VOLUME_MOUNT_PATH,
'/app/data'].filter(Boolean);
```

```
  for (const dir of wanted) {
    try {
      fs.mkdirSync(dir, { recursive: true });
      fs.accessSync(dir, fs.constants.W_OK);
      return dir;
    } catch (_) { /* try the next one */ }
  }
  const local = path.join(__dirname, '..', 'data');
  fs.mkdirSync(local, { recursive: true });
  return local;
}

const DATA_DIR = resolveDataDir();
const UPLOAD_DIR = path.join(DATA_DIR, 'uploads');
fs.mkdirSync(UPLOAD_DIR, { recursive: true });

const db = new DatabaseSync(path.join(DATA_DIR, 'campus-connect.db'));
db.exec(`
  PRAGMA journal_mode = WAL;
  PRAGMA synchronous = NORMAL;
  PRAGMA foreign_keys = ON;
  PRAGMA busy_timeout = 5000;

  CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY,
    username TEXT NOT NULL UNIQUE,
    display_name TEXT NOT NULL,
    password_hash TEXT NOT NULL,
    created_at INTEGER NOT NULL,
    last_seen INTEGER
  );
  CREATE TABLE IF NOT EXISTS sessions (
    token_hash TEXT PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    created_at INTEGER NOT NULL,
    expires_at INTEGER NOT NULL
  );
  CREATE TABLE IF NOT EXISTS conversations (
    id INTEGER PRIMARY KEY,
    kind TEXT NOT NULL CHECK (kind IN ('direct','group')),
    name TEXT,
    direct_key TEXT UNIQUE,
    created_by INTEGER REFERENCES users(id),
    created_at INTEGER NOT NULL
  );
  CREATE TABLE IF NOT EXISTS members (
    conversation_id INTEGER NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    last_read_id INTEGER NOT NULL DEFAULT 0,
    joined_at INTEGER NOT NULL,
    PRIMARY KEY (conversation_id, user_id)
  );
  CREATE TABLE IF NOT EXISTS files (
    id INTEGER PRIMARY KEY,
    conversation_id INTEGER NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    uploader_id INTEGER NOT NULL REFERENCES users(id),
    original_name TEXT NOT NULL,
    stored_name TEXT NOT NULL UNIQUE,
    mime TEXT NOT NULL,
    size INTEGER NOT NULL,
    downloads INTEGER NOT NULL DEFAULT 0,
    created_at INTEGER NOT NULL
  );
  CREATE TABLE IF NOT EXISTS messages (
    id INTEGER PRIMARY KEY,
    conversation_id INTEGER NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    sender_id INTEGER NOT NULL REFERENCES users(id),
    body TEXT NOT NULL DEFAULT '',
    file_id INTEGER REFERENCES files(id),
    created_at INTEGER NOT NULL
  );
  CREATE TABLE IF NOT EXISTS activity (
    id INTEGER PRIMARY KEY,
    kind TEXT NOT NULL,
    user_id INTEGER,
    detail TEXT NOT NULL DEFAULT '',
    created_at INTEGER NOT NULL
  );
  CREATE INDEX IF NOT EXISTS idx_messages_conv ON messages (conversation_id, id);
  CREATE INDEX IF NOT EXISTS idx_members_user ON members (user_id);
  CREATE INDEX IF NOT EXISTS idx_files_conv ON files (conversation_id, id);
  CREATE INDEX IF NOT EXISTS idx_activity_id ON activity (id);
  CREATE INDEX IF NOT EXISTS idx_sessions_exp ON sessions (expires_at);
`);
```

```
// Small helpers so the rest of the code stays short.
const q = {
  get: (sql, ...p) => db.prepare(sql).get(...p),
  all: (sql, ...p) => db.prepare(sql).all(...p),
  run: (sql, ...p) => db.prepare(sql).run(...p),
};
function tx(fn) {
  db.exec('BEGIN IMMEDIATE');
  try {
    const out = fn();
    db.exec('COMMIT');
    return out;
  } catch (err) {
    try { db.exec('ROLLBACK'); } catch (_) { /* already rolled back */ }
    throw err;
  }
}

module.exports = { db, q, tx, DATA_DIR, UPLOAD_DIR };
```

src/metrics.js *counters, round-trip samples and activity log*

```
'use strict';
// Everything here is measured by this server while it runs. Nothing is invented or simulated.
const { q } = require('./db');

const startedAt = Date.now();
const counters = {
  messagesSent: 0,        // messages accepted from clients
  deliveries: 0,          // copies pushed by the server to other connected sockets
  filesUploaded: 0,
  bytesUploaded: 0,
  filesDownloaded: 0,
  bytesDownloaded: 0,
  connectionsOpened: 0,
  disconnections: 0,
  httpRequests: 0,
  httpMsTotal: 0,
  http5xx: 0,
};

// One bucket per minute: messages and uploaded bytes (last 30 minutes kept).
const buckets = new Map();
function minuteKey(t = Date.now()) { return Math.floor(t / 60000) * 60000; }
function bucket() {
  const k = minuteKey();
  if (!buckets.has(k)) {
    buckets.set(k, { t: k, messages: 0, uploadBytes: 0 });
    for (const old of [...buckets.keys()]) if (old < k - 30 * 60000) buckets.delete(old);
  }
  return buckets.get(k);
}

// Live connections (one entry per Socket.IO connection).
const connections = new Map(); // socketId -> { userId, name, username, transport, since, rtt }

// Round-trip times measured by the server (server sends a probe, client answers, server times
it).
const rttSamples = []; // { t, rtt, userId }
function recordRtt(userId, ms) {
  rttSamples.push({ t: Date.now(), rtt: Math.round(ms * 10) / 10, userId });
  if (rttSamples.length > 120) rttSamples.shift();
}

function onMessage(deliveries) {
  counters.messagesSent += 1;
  counters.deliveries += deliveries;
  bucket().messages += 1;
}
function onUpload(bytes) {
  counters.filesUploaded += 1;
  counters.bytesUploaded += bytes;
  bucket().uploadBytes += bytes;
}
function onDownload(bytes) {
  counters.filesDownloaded += 1;
  counters.bytesDownloaded += bytes;
}
function onHttp(ms, status) {
  counters.httpRequests += 1;
  counters.httpMsTotal += ms;
  if (status >= 500) counters.http5xx += 1;
```

```
}

function percentile(sorted, p) {
  if (!sorted.length) return null;
  const i = Math.min(sorted.length - 1, Math.ceil((p / 100) * sorted.length) - 1);
  return sorted[Math.max(0, i)];
}

// ---- Activity log (saved in the database, newest first when read) ----
let activityListener = null;
function onActivity(fn) { activityListener = fn; }
function logActivity(kind, userId, detail = '') {
  const createdAt = Date.now();
  const r = q.run('INSERT INTO activity (kind, user_id, detail, created_at) VALUES (?,?,?,?)',
kind, userId || null, detail, createdAt);
  // keep the table small: remove everything older than the newest 2000 rows
  if (Number(r.lastInsertRowid) % 200 === 0) q.run('DELETE FROM activity WHERE id <= ?',
Number(r.lastInsertRowid) - 2000);
  const row = { id: Number(r.lastInsertRowid), kind, userId: userId || null, detail, createdAt };
  if (activityListener) activityListener(row);
  return row;
}
function recentActivity(limit = 50, kinds = null) {
  const rows = q.all(
    `SELECT a.id, a.kind, a.user_id AS userId, a.detail, a.created_at AS createdAt, u.display_name
AS name, u.username
       FROM activity a LEFT JOIN users u ON u.id = a.user_id
      ORDER BY a.id DESC LIMIT ?`, kinds ? 500 : limit);
  return (kinds ? rows.filter((r) => kinds.includes(r.kind)).slice(0, limit) : rows);
}

function snapshot(extra = {}) {
  const now = Date.now();
  const rtts = rttSamples.map((s) => s.rtt);
  const sorted = [...rtts].sort((a, b) => a - b);
  const all = q.get('SELECT (SELECT COUNT(*) FROM messages WHERE file_id IS NULL) AS messages,
(SELECT COUNT(*) FROM files) AS files, (SELECT COALESCE(SUM(size),0) FROM files) AS bytes');
  const per = [];
  for (let i = 14; i >= 0; i--) {
    const k = minuteKey(now - i * 60000);
    const b = buckets.get(k);
    per.push({ t: k, messages: b ? b.messages : 0, uploadBytes: b ? b.uploadBytes : 0 });
  }
  const transports = { websocket: 0, polling: 0 };
  for (const c of connections.values()) transports[c.transport === 'websocket' ? 'websocket' :
'polling'] += 1;
  return {
    serverTime: now,
    startedAt,
    uptimeSec: Math.floor((now - startedAt) / 1000),
    activeConnections: connections.size,
    sinceStart: {
      ...counters,
      httpAvgMs: counters.httpRequests ? Math.round((counters.httpMsTotal / counters.httpRequests)
* 10) / 10 : 0,
    },
    allTime: { messages: all.messages, files: all.files, bytesStored: all.bytes },
    latency: {
      count: rtts.length,
      last: rtts.length ? rtts[rtts.length - 1] : null,
      avg: rtts.length ? Math.round((rtts.reduce((a, b) => a + b, 0) / rtts.length) * 10) / 10 :
null,
      min: sorted.length ? sorted[0] : null,
      max: sorted.length ? sorted[sorted.length - 1] : null,
      p95: percentile(sorted, 95),
      series: rttSamples.slice(-60).map((s) => ({ t: s.t, rtt: s.rtt })),
    },
    perMinute: per,
    transports,
    connections: [...connections.entries()].map(([id, c]) => ({
      id: id.slice(0, 6), userId: c.userId, name: c.name, username: c.username,
      transport: c.transport, since: c.since, rtt: c.rtt,
    })),
    ...extra,
  };
}

module.exports = {
  counters, connections, startedAt, snapshot, recordRtt, onMessage, onUpload, onDownload, onHttp,
  logActivity, recentActivity, onActivity,
};
```

Appendix B Additional Evidence

TCP sockets. Table B.1 lists the sockets that involved port 3000 while the hand-made WebSocket client of Figure 4.2 was connected. It was read from the kernel's socket table (`/proc/net/tcp`). TIME_WAIT entries are earlier short HTTP requests that had already finished.

Table B.1 TCP sockets involving port 3000 during the handshake test

State	Local address	Remote address
LISTEN	0.0.0.0:3000	0.0.0.0:0
ESTABLISHED	127.0.0.1:3000	127.0.0.1:41390
ESTABLISHED	127.0.0.1:41390	127.0.0.1:3000
TIME_WAIT	127.0.0.1:3000	127.0.0.1:41382
TIME_WAIT	127.0.0.1:40570	127.0.0.1:3000
TIME_WAIT	127.0.0.1:40586	127.0.0.1:3000
TIME_WAIT	127.0.0.1:40600	127.0.0.1:3000
TIME_WAIT	127.0.0.1:40604	127.0.0.1:3000

Complete speed-test statistics. Table B.2 repeats Tables 4.2 and 4.3 with every statistic. Times are in milliseconds; the benchmark ran on 21 October 2026 with Node.js 22.22.0 against `http://127.0.0.1:3098`.

Table B.2 Complete speed-test statistics (milliseconds)

Measurement	n	Min	Mean	Median	95th pct	Max
HTTP `GET /health`	200	0.80	1.51	1.12	3.12	18.51
WebSocket, acknowledgement	100	1.26	1.96	1.74	3.36	10.85
WebSocket, delivery	100	1.17	1.82	1.65	2.63	11.02
Long-polling, acknowledgement	40	3.62	7.67	7.42	11.74	13.63
Long-polling, delivery	40	2.99	5.02	4.90	8.06	8.53
Group of 10, last receiver	20	1.48	3.08	2.00	14.74	14.74
1 MB upload	3	7.58	15.03	14.52	23.00	23.00
1 MB download	3	4.44	7.87	5.15	14.02	14.02
5 MB upload	3	31.77	39.16	42.13	43.60	43.60
5 MB download	3	12.50	16.61	13.90	23.42	23.42
15 MB upload	3	87.95	90.92	91.57	93.25	93.25
15 MB download	3	72.12	79.53	76.24	90.24	90.24

Activity log and fallback. Figure B.1 shows the Activity page after the session of Figure 3.2. For both students the log has "created an account", then "connected over polling", then

“upgraded from polling to websocket”, as the Socket.IO documentation describes [4]. Message text and file names are never written to this log.

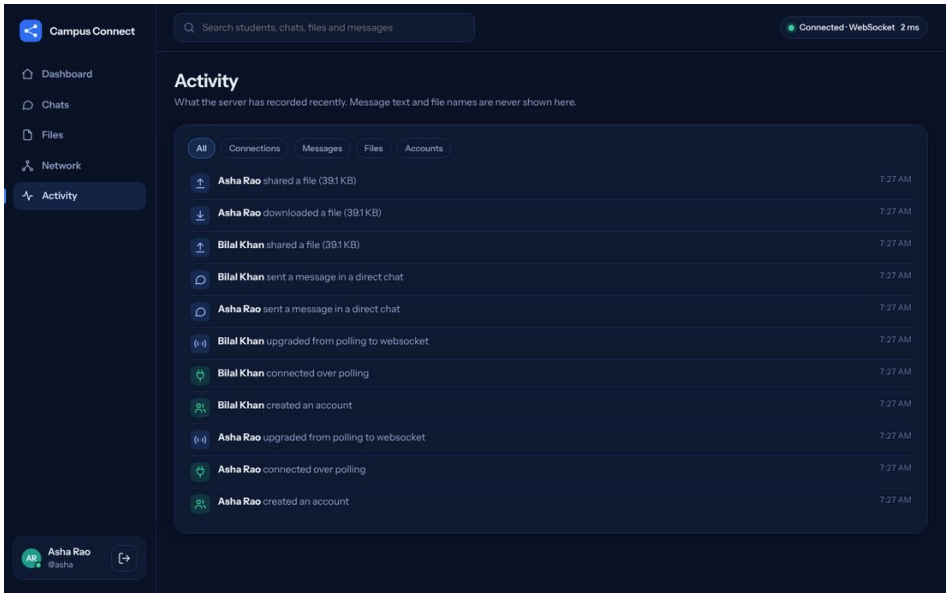


Figure B.1 Activity page showing connection upgrades and transfers

Reconnection. The browser of Student A was put into offline mode with the automation tool and then back online. Figure B.2 shows the page while offline: the badge reads “Cannot reach the server. Retrying” and the connection panel says “Reconnecting”. Figure B.3 shows the page after the connection returned: the badge reads “Connected · HTTP long-polling” and a notice says “Back online. Catching up on anything you missed.” The client had reconnected through long-polling, which it upgrades to WebSocket afterwards. The connection panel in Figure B.2 still names the previous transport while reconnecting; this is cosmetic.

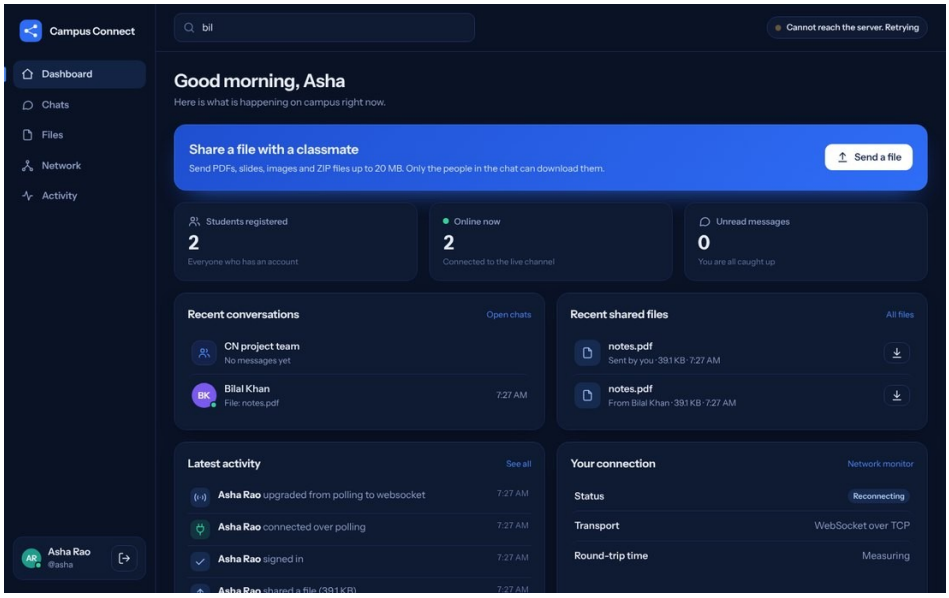


Figure B.2 Browser while the connection to the server was cut

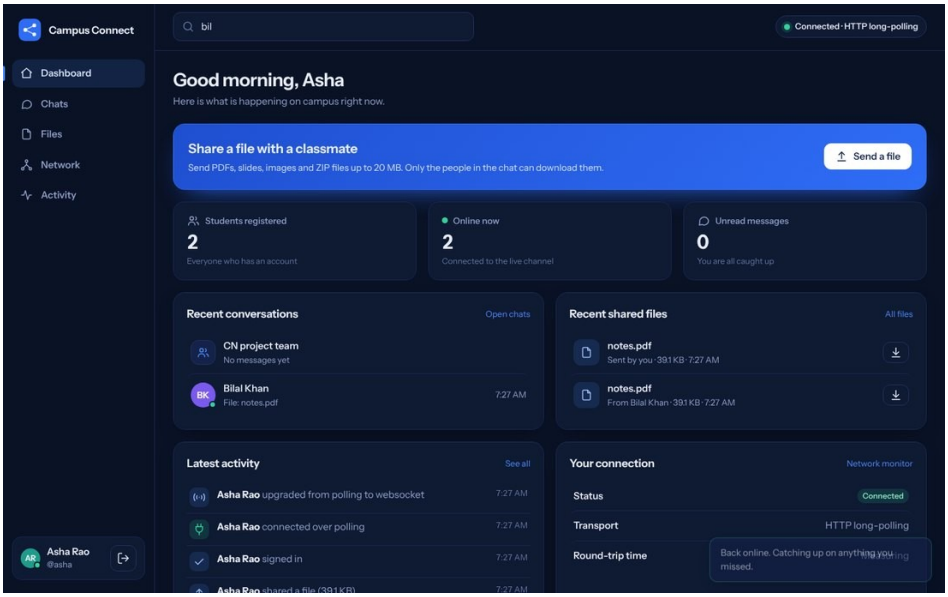


Figure B.3 Browser after the connection returned

Complete output of npm test *21 October 2026, Node.js 22.22.0 (the port number differs on every run)*

```
Campus Connect tests (server on port 42753)

Health and protection
  ok   health endpoint answers ok
  ok   security headers are set
  ok   chat API needs a login
  ok   file download needs a login
  ok   live channel refuses visitors who are not signed in
  ok   requests from another website are blocked

Accounts
  ok   register works
  ok   session cookie is HttpOnly and SameSite=Lax
  ok   register again with the same username is refused
  ok   short password is refused
  ok   bad username is refused
  ok   three students registered
  ok   passwords are not stored as plain text
  ok   wrong password is refused
  ok   login works
  ok   /api/me returns the signed-in student

Live messaging
  ok   online status shows connected students
  ok   direct chat created
  ok   the other student is told about the new chat
  ok   same pair gets the same chat (no duplicates)
  ok   message is acknowledged
  ok   recipient receives it instantly with sender and time
  ok   unread count is 1 for the recipient
  ok   marking as read clears the unread count
  ok   empty message is refused
  ok   too long message is refused
  ok   a student outside the chat cannot post in it
  ok   a student outside the chat cannot read it
  ok   history is stored and returned
  ok   group chat created with three members
  ok   group message reaches every member

File sharing
  ok   PDF upload accepted
  ok   recipient is notified live with file name and size
  ok   recipient can download the same bytes
  ok   download is sent as an attachment and not sniffed
  ok   student outside the chat cannot download it
  ok   file ids that do not exist look the same (404)
  ok   fake PDF (text renamed .pdf) is rejected
  ok   program files (.exe) are rejected
  ok   files over the limit are refused (413)
```

```
  ok   outsider cannot upload into a chat
  ok   folder tricks in a file name are cleaned
  ok   files are stored outside the web folder with random names
  ok   names with accents and other scripts are kept
  ok   file list shows files shared with the student
  ok   dashboard totals are right

Search
  ok   search finds students by name
  ok   search finds files by name
  ok   search finds messages
  ok   search never shows files from other people's chats

Network monitor (real measurements)
  ok   active connections counted
  ok   messages counted
  ok   uploads and bytes counted
  ok   download counted
  ok   latency was measured by the server
  ok   transport types are reported
  ok   activity log has connections, messages, uploads and downloads
  ok   monitor pushes live snapshots
  ok   students see someone go offline
  ok   disconnect is logged with the reason

Polling fallback and reconnection
  ok   HTTP long-polling works when WebSocket is not used
  ok   messages flow over the fallback too
  ok   a dropped connection reconnects by itself

Restart: saved data survives
  ok   login still works after restart
  ok   messages are still there
  ok   uploaded file is still downloadable and identical
  ok   existing sign-in still valid after restart
  ok   logout ends the session
  ok   private data is refused after logout

Sign-in rate limit
  ok   too many sign-in attempts get a 429

70 checks passed, 0 failed
exit 0
```

Appendix C Individual Contributions

This project has a single member, so Table C.1 has one row.

Table C.1 Individual contributions

Student Name and PRN	Work Completed	Evidence or Deliverable
Harsh Patil	Requested the project and supplied the course template; attempted the public deployment on Railway (Section 5.4); submits this report. The design, code, tests, measurements, diagrams and report text were produced with the AI assistant described in the AI Usage Declaration.	Project folder `campus-connect`; this report; the Railway attempt described in Table 5.2